```
extends Node
const Hall = preload("res://scripts/hall.gd")
const LayeredHall = preload("res://scripts/layered_hall.gd")
const Entrance = preload("res://scripts/entrance.gd")
const CourtCopy = preload("res://scripts/court_copy.gd")
const SWITCH_TINT := Color("#3a1d12")
const COURT_FROM_HALL := Vector2(768, 260)
const Backhall = preload("res://scripts/backhall.gd")
const BACKHALL_START := "res://scripts/backhall_start.gd"
const BackhallUI = preload("res://scripts/backhall_ui.gd")
const BACKHALL_FROM_HALL := Vector2(100, 780)
const HALL_FROM_BACKHALL := Vector2(1440, 600)
const Lawn = preload("res://scripts/lawn.gd")
const LawnUI = preload("res://scripts/lawn_ui.gd")
const LAWN_FROM_COURT := Vector2(100, 780)
const LAWN_FROM_BACKHALL := Vector2(100, 300)
const COURT_FROM_LAWN := Vector2(1440, 780)
const Lake = preload("res://scripts/lake.gd")
const LakeUI = preload("res://scripts/lake_ui.gd")
const LAKE_FROM_COURT := Vector2(1440, 956)
const COURT_FROM_LAKE := Vector2(100, 520)
const Orchard = preload("res://scripts/orchard.gd")
const OrchardUI = preload("res://scripts/orchard_ui.gd")
const OrchardSave = preload("res://scripts/orchard_save.gd")
const ORCHARD_FROM_LAWN := Vector2(768, 1076)
const LAWN_FROM_ORCHARD := Vector2(768, 220)
const BACKHALL_FROM_LAWN := Vector2(1440, 780)
var court
var backhall
var backhall_ui: Control
var lawn
var lawn_ui: Control
var lake
var lake_ui: Control
var orchard
var orchard_ui: Control
var orchard_save
const BanquetUI = preload("res://scripts/banquet_ui.gd")
var banquet_ui: Control
var add_dish_btn: Button
var kids_heard := false
var cv
var switching := false
var hall_door_latched := true
var hall_side_latched := true
var profile_saved := false
const CourtSave = preload("res://scripts/court_save.gd")
const CourtUI = preload("res://scripts/court_ui.gd")
var court_save
var court_ui: Control
```

```
var photo_greeted := false
var rest_heard := false
var debug_override := false
const USE_LAYERED := true
const Spr = preload("res://scripts/sprites.gd")
const LINE_ARRIVE := "到啦～ 罗曼山庄 席还没散"
const LINE_TABLE := "想跟我们说点什么 写这儿 只有我们看得到"
const LINE_THANKS := "收到啦 谢谢你专门来一趟"
const LINE_TOASTED_FMT := "敬了 %s 一杯"
const LINE_CHEERS := "干杯"
const LINE_MEET := "想认识你"
const LINE_MEET_SENT_FMT := "想认识 %s 记下了"
const LINE_BENCH := "桌子都坐满了 先坐墙边的长凳"
const LINE_FIRST_WITH_INVITER := "%s 带你来的 你们坐一桌"
const LINE_FIRST_INVITER_OTHER := "%s 带你来的 你坐 %s 那桌"
const LINE_FIRST_INVITER_NEW := "%s 带你来的 这桌还空着 你先坐"
const LINE_FIRST_EMPTY := "这桌还空着 你是第一个坐的"
const LINE_FIRST_TABLE := "你坐 %s 那桌"
const ArrivalCard = preload("res://scripts/arrival_card.gd")
var arrival_card: Control = null
const ToastScene = preload("res://scripts/toast_scene.gd")
const GameCfg = preload("res://scripts/games/game_config.gd")
const GameSave = preload("res://scripts/games/game_save.gd")
const GameHost = preload("res://scripts/games/game_host.gd")
const LINE_SIT_FIRST_GAME := "先坐下 再玩"
const LINE_GIFT_DONE_FMT := "添了 %d 金 礼金簿记上了"
const GIFT_CHOICES := [8, 88, 188, 888]
const LINE_GIFT_BTN_FMT := "%d 金"
const LINE_GIFT_ALL_FMT := "全部 %d 金"
const LINE_GIFT_ASK_FMT := "%d 金全添上？"
const LINE_GIFT_YES := "添"
var games_save
var game_host: Control
var play_btn: Button
var coins_chip: Label
var gift_row: HBoxContainer
var gift_pick: HBoxContainer
const PULSE_EVERY := 8.0
var pulse_t := 0.0
var pulse_since := 0
var pulse_busy := false
var hall_ready := false
var pulse_seen: Dictionary = {}
var toast_scene: Control
var toast_queue: Array = []
const LINE_TOAST_BACK := "回敬一杯"
const TOAST_BACK_SECS := 6.0
var toast_back_btn: Button
var toast_back_to := ""
var toast_back_info: Dictionary = {}
```

```
var toast_back_note := -1
const LINE_BOOK_EMPTY := "还没人写 你来第一句"
const BOOK_PLACEHOLDER := "接一句"
const LINE_BOOK_READONLY := "先坐下 慢慢写"
const GOLD := Color("#d4af37")
const GOLD1 := Color("#f4d97a")
const RED := Color("#c0392b")
const RED0 := Color("#8c2f27")
const PAPER := Color("#efe6d8")
const MUTED := Color("#b1a6ba")
const INK := Color("#15111b")
const FONT_PATH := "res://assets/fonts/fusion-pixel-12px-proportional-zh_hans.otf.woff2"
var font: FontFile
var ui: Control
var view: SubViewportContainer
var hall
var fade: ColorRect
var screens := {}
var hud: Control
var dialog: PanelContainer
var dialog_text: Label
var dialog_who: Label
var dialog_gloss := ""
var say_log: Array = []
var hint: PanelContainer
var hint_done := false
const LINE_WALK_HINT := "点地面走路\n或推左下圆圈"
const LINE_LOOK_FMT := "看看 %s"
var note: Label
var note_box: PanelContainer
var note_token := 0
const TOP_Y := 150.0
var sit_btn: Button
var sit_target := {"table_id": "", "chair_no": 0}
var sit_confirming := false
var person_btn: Button
var near_seat_info := {"table_id": "", "chair_no": 0}
var near_person_id := ""
var card_open := false
var card_openid := ""
var person_card: Control
var person_card_portrait: TextureRect
var person_card_name: Label
var person_card_inviter: Label
var person_card_toast_btn: Button
var person_card_meet_btn: Button
var book_btn: Button
var rel_btn: Button
var near_rel := {"title": "", "relation": ""}
const LINE_REL_FMT := "看看%s"
```

```
var feast_force := ""
const FEAST_NAMES := {"": "按北京时间", "feast": "强制开席", "pre": "强制开席前"}
var chat_btn: Button
var near_family := {"key": "", "title": ""}
const LINE_CHAT_FMT := "和%s聊聊"
const MAIN_LINES_PATH := "res://assets/main_table_lines.json"
var story_btn: Button
var story_card: Control
var story_open := false
var near_table_id := ""
var book_open := false
var book_table := ""
var book_empty_shown := false
var book_panel: Control
var book_list_box: VBoxContainer
var book_scroll: ScrollContainer
var book_loading: Label
var book_input_row: VBoxContainer
var book_readonly_label: Label
var book_edit: TextEdit
var book_count: Label
var book_err: Label
var book_ok: Button
const CloudRpc = preload("res://scripts/cloud.gd")
var cloud: Node
const REASONS := {
    "seat_taken": "这把有人坐了",
    "main_table_only_family": "主桌是阿鱼家里人坐的",
    "toast_limit": "今天敬够了", "toast_same_person": "这杯敬过了",
    "toast_self": "自己就不用敬了", "not_sat_here": "先坐下 慢慢写",
    "book_too_long": "太长啦 真实情感的话 短一点也没关系",
    "network": "没连上 再试一下", "no_resident": "没连上 再试一下",
    "bad_request": "没连上 再试一下", "no_seat": "没连上 再试一下",
    "sensitive": "换一个吧", "reserved": "这个名字留给主角了，换一个吧",
    "meet_self": "自己就不用认识了", "meet_already": "这句说过了", "meet_not_met": "先坐到一桌 或者
敬一杯",
}
var nick_edit: LineEdit
var nick_err: Label
var nick_count: Label
var nick_ok: Button
var whisper_box: PanelContainer
var whisper_edit: TextEdit
var whisper_count: Label
var whisper_ok: Button
var state := ""
var nickname := ""
var preset := 2
var fast := false
var say_token := 0
```

```
const STICK_R := 64.0
var stick_base: Panel
var stick_knob: Panel
var stick_touch := -1
var stick_origin := Vector2.ZERO
var touches := {}
var dbg: Button
var dbg_panel: PanelContainer
var dbg_text: Label
var dbg_acc := 0.0
var dbg_log_acc := 0.0
var dbg_worst := 0.0
const RemoteProbe = preload("res://scripts/remote_probe.gd")
var remote_probe: RefCounted
var remote_status: Label
var remote_img: TextureRect
const Hall55 = preload("res://scripts/hall55_test.gd")
var h55_view: SubViewportContainer
var h55: Node2D
var h55_info: Label
var h55_box: PanelContainer
var h55_acc := 0.0
func _ready() -> void:
    print("[suqi] 脚本开始")
    font = _make_font()
    print("[suqi] 字体好了")
    ui = Control.new()
    ui.set_anchors_preset(Control.PRESET_FULL_RECT)
    ui.mouse_filter = Control.MOUSE_FILTER_IGNORE
    ui.theme = _theme()
    add_child(ui)
    view = SubViewportContainer.new()
    view.stretch = true
    view.set_anchors_preset(Control.PRESET_FULL_RECT)
    view.mouse_filter = Control.MOUSE_FILTER_IGNORE
    if not USE_LAYERED:
        var pm := ShaderMaterial.new()
        pm.shader = load("res://shaders/post.gdshader")
        view.material = pm
    ui.add_child(view)
    var sv := SubViewport.new()
    sv.msaa_3d = Viewport.MSAA_DISABLED
    view.add_child(sv)
    cloud = CloudRpc.new()
    add_child(cloud)
    games_save = GameSave.new()
    games_save.load_local()
    games_save.saver = _save_games
    _make_hall(sv)
    court = Entrance.new()
```

```
    sv.add_child(court)
    court.build(preset, "", font)
    court.spot.connect(_on_court_spot)
    court.walking_enabled = false
    print("[suqi] 迎宾院搭好了")
    backhall = load(BACKHALL_START).new() if "--backhall=start" in OS.get_cmdline_user_args() else B
ackhall.new()
    sv.add_child(backhall)
    backhall.build(preset, "", font)
    backhall.spot.connect(_on_backhall_spot)
    backhall.walking_enabled = false
    print("[suqi] 后厅搭好了")
    lawn = Lawn.new()
    sv.add_child(lawn)
    lawn.build(preset, "", font)
    lawn.spot.connect(_on_lawn_spot)
    lawn.fetched.connect(_on_lawn_fetched)
    lawn.walking_enabled = false
    print("[suqi] 草坪搭好了")
    lake = Lake.new()
    sv.add_child(lake)
    lake.build(preset, "", font)
    lake.spot.connect(_on_lake_spot)
    lake.walking_enabled = false
    print("[suqi] 湖边搭好了")
    orchard_save = OrchardSave.new()
    orchard_save.load_local()
    orchard = Orchard.new()
    sv.add_child(orchard)
    orchard.build(preset, "", font)
    orchard.save = orchard_save
    orchard.refresh_fruit()
    orchard.spot.connect(_on_orchard_spot)
    orchard.walking_enabled = false
    print("[suqi] 果园搭好了")
    _show_canvas("banquet")
    _build_ui()
    court_save = CourtSave.new()
    court_save.load_local()
    court_ui = CourtUI.new()
    ui.add_child(court_ui)
    court_ui.setup(self, font, court_save)
    court_ui.panel_changed.connect(func(open): court.walking_enabled = state == "court" and not open
)
    backhall_ui = BackhallUI.new()
    ui.add_child(backhall_ui)
    backhall_ui.setup(self)
    lawn_ui = LawnUI.new()
    ui.add_child(lawn_ui)
    lawn_ui.setup(self)
```

```
    lake_ui = LakeUI.new()
    ui.add_child(lake_ui)
    lake_ui.setup(self)
    orchard_ui = OrchardUI.new()
    ui.add_child(orchard_ui)
    orchard_ui.setup(self)
    banquet_ui = BanquetUI.new()
    ui.add_child(banquet_ui)
    banquet_ui.setup(self, font)
    stories = load_stories(STORIES_PATH)
    banquet_ui.panel_changed.connect(func(open):
        hall.walking_enabled = state == "banquet" and not open and not whisper_box.visible
        _refresh_seat_buttons())
    ui.move_child(fade, -1)
    print("[suqi] 界面搭好了")
    var cap := ""
    var shot := ""
    var start := ""
    for a in OS.get_cmdline_user_args():
        if a.begins_with("--capture="): cap = a.substr(10)
        elif a.begins_with("--shot="): shot = a.substr(7)
        elif a.begins_with("--start="): start = a.substr(8)
    if cap != "":
        fast = true
        if "--hud" in OS.get_cmdline_user_args():
            _build_debug()
        await _setup_shot(shot)
        for i in 50: await get_tree().process_frame
        await RenderingServer.frame_post_draw
        get_viewport().get_texture().get_image().save_png(cap.path_join(shot + ".png"))
        print("CAPTURED ", shot, " ", _render_stats())
        get_tree().quit()
    else:
        fade.color.a = 1.0
        var wait := _label("加载中", 24, MUTED)
        wait.set_anchors_and_offsets_preset(Control.PRESET_FULL_RECT)
        wait.vertical_alignment = VERTICAL_ALIGNMENT_CENTER
        ui.add_child(wait)
        await hall.warm_up()
        wait.queue_free()
        print("[suqi] 着色器预热完了")
        if OS.has_feature("suqi_debug") or "--hud" in OS.get_cmdline_user_args():
            _build_debug()
        if start != "":
            await _setup_shot(start)
            fade.color.a = 0.0
            print("[suqi] 直接到了 ", start, " (--start) ")
            return
        _goto("title")
        _fade_to(0.0)
```

```
        get_tree().create_timer(2.0).timeout.connect(func():
            if state == "title" and fade.color.a > 0.0:
                fade.color.a = 0.0
                print("[suqi] 开场黑幕是被保险掀开的"))
        await get_tree().process_frame
        print("[suqi] 开场出来了")
func _sb(bg: Color, border: Color, bw := 2, rad := 6, pad := 14) -> StyleBoxFlat:
    var s := StyleBoxFlat.new()
    s.bg_color = bg
    s.border_color = border
    s.set_border_width_all(bw)
    s.set_corner_radius_all(rad)
    s.content_margin_left = pad; s.content_margin_right = pad
    s.content_margin_top = pad * 0.7; s.content_margin_bottom = pad * 0.7
    return s
func _build_debug() -> void:
    dbg = Button.new()
    dbg.add_theme_font_size_override("font_size", 24)
    dbg.add_theme_color_override("font_color", Color("#9CFF9C"))
    dbg.add_theme_color_override("font_hover_color", Color("#9CFF9C"))
    dbg.add_theme_color_override("font_pressed_color", Color("#9CFF9C"))
    for st in ["normal", "hover", "pressed", "focus"]:
        dbg.add_theme_stylebox_override(st, _sb(Color(0, 0, 0, 0.55), Color(0, 0, 0, 0), 0, 4, 8))
    dbg.position = Vector2(12, 100)
    dbg.text = "测帧率中"
    dbg.pressed.connect(_toggle_log)
    ui.add_child(dbg)
    var spd := Button.new()
    spd.add_theme_font_size_override("font_size", 24)
    spd.add_theme_color_override("font_color", Color("#9CFF9C"))
    for st in ["normal", "hover", "pressed", "focus"]:
        spd.add_theme_stylebox_override(st, _sb(Color(0, 0, 0, 0.55), Color(0, 0, 0, 0), 0, 4, 8))
    spd.set_anchors_preset(Control.PRESET_BOTTOM_RIGHT)
    spd.offset_left = -170; spd.offset_right = -12
    spd.offset_top = -60; spd.offset_bottom = -16
    spd.text = "走路 %.1f" % hall.speed_m if USE_LAYERED else "走路"
    spd.pressed.connect(func():
        if not USE_LAYERED:
            return
        var i: int = hall.SPEED_CHOICES.find(hall.speed_m)
        hall.speed_m = hall.SPEED_CHOICES[(i + 1) % hall.SPEED_CHOICES.size()]
        spd.text = "走路 %.1f" % hall.speed_m
        print("[suqi] 走路速度切到 ", hall.speed_m, " 米/秒"))
    ui.add_child(spd)
    dbg_panel = PanelContainer.new()
    dbg_panel.set_anchors_preset(Control.PRESET_FULL_RECT)
    dbg_panel.offset_left = 10; dbg_panel.offset_right = -10
    dbg_panel.offset_top = 150; dbg_panel.offset_bottom = -10
    var dv := VBoxContainer.new()
    dv.add_theme_constant_override("separation", 8)
```

```
    dbg_panel.add_child(dv)
    var rrow := HBoxContainer.new()
    rrow.add_theme_constant_override("separation", 8)
    var rb := Button.new()
    rb.text = "远程加载"
    rb.pressed.connect(_run_remote_probe)
    rrow.add_child(rb)
    var rc := Button.new()
    rc.text = "删掉本地副本"
    rc.pressed.connect(_clear_remote_probe)
    rrow.add_child(rc)
    dv.add_child(rrow)
    var h55b := Button.new()
    h55b.text = "55 桌试验厅"
    h55b.pressed.connect(_open_hall55.bind(6))
    dv.add_child(h55b)
    var bigb := Button.new()
    bigb.name = "hall55_toggle"
    bigb.text = "宴会厅：%s（点一下换）" % HALL_VARIANT_NAMES.get(hall_variant(), "")
    bigb.pressed.connect(func():
        var vs: Array = hall_variants()
        var i: int = vs.find(hall_variant())
        var nv: String = vs[(i + 1) % vs.size()]
        dbg_panel.visible = false
        await _set_hall_variant(nv)
        bigb.text = "宴会厅：%s（点一下换）" % HALL_VARIANT_NAMES.get(nv, ""))
    dv.add_child(bigb)
    var feastb := Button.new()
    feastb.name = "feast_toggle"
    feastb.text = "亲友：%s（点一下换）" % FEAST_NAMES.get(feast_force, "")
    feastb.pressed.connect(func():
        var order := ["", "feast", "pre"]
        var nf: String = order[(order.find(feast_force) + 1) % order.size()]
        _set_feast_force(nf)
        feastb.text = "亲友：%s（点一下换）" % FEAST_NAMES.get(nf, ""))
    dv.add_child(feastb)
    remote_status = _label("", 24, GOLD1, HORIZONTAL_ALIGNMENT_LEFT)
    dv.add_child(remote_status)
    remote_img = TextureRect.new()
    remote_img.custom_minimum_size = Vector2(0, 240)
    remote_img.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    remote_img.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    remote_img.visible = false
    dv.add_child(remote_img)
    var sc := ScrollContainer.new()
    sc.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    sc.size_flags_vertical = Control.SIZE_EXPAND_FILL
    dbg_text = _label("", 12, PAPER, HORIZONTAL_ALIGNMENT_LEFT)
    dbg_text.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    sc.add_child(dbg_text)
```

```
    dv.add_child(sc)
    dbg_panel.visible = false
    ui.add_child(dbg_panel)
func _toggle_log() -> void:
    dbg_panel.visible = not dbg_panel.visible
    if dbg_panel.visible:
        dbg_text.text = _read_js_log()
func _run_remote_probe(p: RefCounted = null) -> void:
    if remote_probe != null:
        return
    remote_status.text = "远程加载中"
    remote_probe = p if p != null else RemoteProbe.new()
    remote_probe.run(cloud, func(r: Dictionary):
        remote_probe = null
        remote_status.text = str(r.get("text", ""))
        var tex = r.get("texture", null)
        remote_img.texture = tex
        remote_img.visible = tex != null
        dbg_text.text = _read_js_log())
func _clear_remote_probe() -> void:
    if remote_probe != null:
        return
    var had: bool = RemoteProbe.new().clear_local()
    remote_status.text = "本地副本删了，再点「远程加载」会重新下载" if had else "本地没有副本"
    remote_img.texture = null
    remote_img.visible = false
func _read_js_log() -> String:
    if not (OS.has_feature("web") or OS.has_feature("wechat")):
        return "电脑上没有这份日志，看 Godot 的输出窗口"
    var caps := ""
    if cloud != null:
        caps = cloud.diagnose() + "\n\n"
    var log_obj = JavaScriptBridge.get_interface("__suqiLog")
    if log_obj == null:
        return caps + "读不到日志：调试脚本没加载"
    return caps + str(log_obj.text())
func _process(dt: float) -> void:
    if dialog != null:
        _layout_top()
    if coins_chip != null and _in_scene():
        coins_chip.text = "%d 金" % games_save.coins
    if state == "banquet" and not switching and hall.door_r > 0.0:
        var dd: float = hall.pos.distance_to(hall.door)
        if hall_door_latched:
            if dd > hall.door_r:
                hall_door_latched = false
        elif dd < hall.door_r and hall.walking_enabled:
            _switch_canvas("court")
    if state == "banquet" and not switching and USE_LAYERED and hall.side_door_r > 0.0:
        var sd: float = hall.pos.distance_to(hall.side_door)
```

```
        if hall_side_latched:
            if sd > hall.side_door_r:
                hall_side_latched = false
        elif sd < hall.side_door_r and hall.walking_enabled:
            _switch_canvas("backhall")
    if state == "banquet" and hall_ready and not pulse_busy and h55 == null:
        pulse_t += dt
        if pulse_t >= (1.0 if pulse_since == 0 else PULSE_EVERY):
            pulse_t = 0.0
            pulse_busy = true
            cloud.call_fn("hallPulse", {"since": pulse_since}, _on_pulse)
    if state == "banquet" and banquet_ui != null:
        banquet_ui.sync()
    if state == "lake" and lake_ui != null:
        lake_ui.sync()
    if state == "orchard" and orchard_ui != null:
        orchard_ui.sync()
    if stick_base != null:
        var show_stick: bool = _in_scene() and cv.walking_enabled
        stick_base.visible = show_stick
        stick_knob.visible = show_stick
        if not show_stick and stick_touch != -1:
            stick_touch = -1
            cv.stick = Vector2.ZERO
            _stick_rest()
    if dbg == null:
        return
    if h55 != null:
        h55_acc += dt
        if h55_acc >= 0.5:
            h55_acc = 0.0
            h55_info.text = h55.report()
            if ui.get_child(ui.get_child_count() - 1) != dbg:
                ui.move_child(h55_view, -1)
                ui.move_child(h55_box, -1)
                ui.move_child(dbg, -1)
    dbg_worst = maxf(dbg_worst, dt)
    if dt > 0.15:
        print("[suqi] 卡了 %d 毫秒（%s）" % [int(dt * 1000.0), state])
    dbg_acc += dt
    if dbg_acc >= 0.5:
        dbg.text = "%d 帧/秒  最慢 %d 毫秒  绘制 %d" % [Engine.get_frames_per_second(), int(dbg_wors
t * 1000.0),
            RenderingServer.get_rendering_info(RenderingServer.RENDERING_INFO_TOTAL_DRAW_CALLS_IN_FR
AME)]
        dbg_acc = 0.0
        dbg_worst = 0.0
    dbg_log_acc += dt
    if dbg_log_acc >= 5.0:
        dbg_log_acc = 0.0
```

```
        print("[suqi] " + dbg.text)
func _open_hall55(per: int) -> void:
    _close_hall55()
    dbg_panel.visible = false
    view.visible = false
    view.get_child(0).process_mode = Node.PROCESS_MODE_DISABLED
    h55_view = SubViewportContainer.new()
    h55_view.stretch = true
    h55_view.set_anchors_preset(Control.PRESET_FULL_RECT)
    h55_view.mouse_filter = Control.MOUSE_FILTER_STOP
    ui.add_child(h55_view)
    var sv := SubViewport.new()
    h55_view.add_child(sv)
    h55 = Hall55.new()
    h55.per = per
    sv.add_child(h55)
    h55_box = PanelContainer.new()
    h55_box.add_theme_stylebox_override("panel", _sb(Color(0, 0, 0, 0.7), Color(0, 0, 0, 0), 0, 8, 1
2))
    h55_box.position = Vector2(12, 160)
    var v := VBoxContainer.new()
    v.add_theme_constant_override("separation", 10)
    h55_box.add_child(v)
    h55_info = _label(h55.report(), 24, Color("#9CFF9C"), HORIZONTAL_ALIGNMENT_LEFT)
    v.add_child(h55_info)
    var row := HBoxContainer.new()
    row.add_theme_constant_override("separation", 10)
    for b in [["330 人", func(): _open_hall55(6)], ["550 人", func(): _open_hall55(10)], ["回去", _c
lose_hall55]]:
        var btn := Button.new()
        btn.text = b[0]
        btn.add_theme_font_size_override("font_size", 26)
        btn.custom_minimum_size = Vector2(120, 64)
        btn.pressed.connect(b[1])
        row.add_child(btn)
    v.add_child(row)
    ui.add_child(h55_box)
    ui.move_child(h55_view, -1)
    ui.move_child(h55_box, -1)
    print("[suqi] 打开 55 桌试验厅，每桌 ", per, " 人")
func _close_hall55() -> void:
    if h55_view == null:
        return
    h55_view.queue_free()
    h55_box.queue_free()
    h55_view = null
    h55_box = null
    h55 = null
    view.visible = true
    view.get_child(0).process_mode = Node.PROCESS_MODE_INHERIT
```

```
    print("[suqi] 关了 55 桌试验厅")
const HALL_CFG := "res://assets/hall.json"
const HALL55_FLAG := "user://hall55.flag"
const HALL_VARIANTS := ["", "v11", "55"]
const HALL_VARIANT_NAMES := {"": "第 44 版", "v11": "照效果图（第 46 版）", "55": "55 桌大厅"}
static func hall_variants() -> Array:
    var out: Array = []
    for v in HALL_VARIANTS:
        if v != "v11" or FileAccess.file_exists(LayeredHall.DATA_V11):
            out.append(v)
    return out
func hall_variant() -> String:
    if hall_variant_once != "?":
        return hall_variant_once
    if FileAccess.file_exists(HALL55_FLAG):
        var v := FileAccess.get_file_as_string(HALL55_FLAG).strip_edges()
        if v == "1":
            return "55"
        if v == "44":
            return ""
        if v != "" and v in hall_variants():
            return v
    if FileAccess.file_exists(HALL_CFG):
        var j = JSON.parse_string(FileAccess.get_file_as_string(HALL_CFG))
        if j is Dictionary and bool(j.get("hall55", false)):
            return "55"
    return ""
func hall55_on() -> bool:
    return hall_variant() == "55"
func _make_hall(sv: Node) -> void:
    hall = LayeredHall.new() if USE_LAYERED else Hall.new()
    if USE_LAYERED:
        match hall_variant():
            "55":
                hall.data_path = LayeredHall.DATA_55
            "v11":
                hall.data_path = LayeredHall.DATA_V11
    sv.add_child(hall)
    sv.move_child(hall, 0)
    hall.build(preset, "", font)
    print("[suqi] 宴会厅搭好了（", HALL_VARIANT_NAMES.get(hall_variant(), ""), "）")
    hall.reached.connect(_on_reached)
    if USE_LAYERED:
        hall.chair_near.connect(_on_chair_near)
        hall.person_near.connect(_on_person_near)
        hall.table_near.connect(_on_table_near)
        hall.game_tap.connect(_on_game_tap)
        cloud.fake_tables = (hall.cfg.get("tables", []) as Array).size()
        cloud.fake_table_ids = []
        near_family = {"key": "", "title": ""}
```

```
        if hall.life != null and hall.life.has_signal("family_near"):
            hall.life.family_near.connect(_on_family_near)
        near_rel = {"title": "", "relation": ""}
        if hall.life != null and hall.life.relatives != null:
            hall.life.relatives.rel_near.connect(_on_rel_near)
            hall.life.relatives.set_force(feast_force)
        if rel_btn != null:
            _refresh_seat_buttons()
        if hall.big:
            for t in hall.cfg.get("tables", []):
                cloud.fake_table_ids.append(str(t["id"]))
    hall.walking_enabled = false
var hall_variant_once := "?"
func _set_hall55(on: bool, keep := true) -> void:
    await _set_hall_variant("55" if on else "", keep)
func _set_hall_variant(v: String, keep := true) -> void:
    if keep:
        var f := FileAccess.open(HALL55_FLAG, FileAccess.WRITE)
        if f != null:
            f.store_string(v if v != "" else "44")
            f.close()
    else:
        hall_variant_once = v
    var sv: Node = hall.get_parent()
    var was_banquet := state == "banquet"
    hall.queue_free()
    hall = null
    _make_hall(sv)
    if was_banquet:
        await _enter_banquet()
    else:
        hall.visible = false
        hall.process_mode = Node.PROCESS_MODE_DISABLED
    print("[suqi] 宴会厅切到 ", HALL_VARIANT_NAMES.get(v, v))
func _render_stats() -> String:
    return "绘制 %d 次 物体 %d 个 三角形 %d" % [
        RenderingServer.get_rendering_info(RenderingServer.RENDERING_INFO_TOTAL_DRAW_CALLS_IN_FRAME)
,
        RenderingServer.get_rendering_info(RenderingServer.RENDERING_INFO_TOTAL_OBJECTS_IN_FRAME),
        RenderingServer.get_rendering_info(RenderingServer.RENDERING_INFO_TOTAL_PRIMITIVES_IN_FRAME)
]
func _make_font() -> FontFile:
    var f := load(FONT_PATH) as FontFile
    f.antialiasing = TextServer.FONT_ANTIALIASING_NONE
    f.hinting = TextServer.HINTING_NONE
    f.subpixel_positioning = TextServer.SUBPIXEL_POSITIONING_DISABLED
    f.fixed_size = 12
    f.fixed_size_scale_mode = TextServer.FIXED_SIZE_SCALE_INTEGER_ONLY
    return f
func _theme() -> Theme:
```

```
    var t := Theme.new()
    t.default_font = font
    t.default_font_size = 24
    t.set_color("font_color", "Label", PAPER)
    t.set_stylebox("panel", "PanelContainer", _sb(Color(0.106, 0.078, 0.133, 0.94), GOLD))
    t.set_stylebox("normal", "Button", _sb(RED0, GOLD, 2, 6, 18))
    t.set_stylebox("hover", "Button", _sb(RED, GOLD1, 2, 6, 18))
    t.set_stylebox("pressed", "Button", _sb(Color("#6e211b"), GOLD1, 2, 6, 18))
    t.set_stylebox("disabled", "Button", _sb(Color("#3b3148"), Color("#5a4e68"), 2, 6, 18))
    t.set_stylebox("focus", "Button", _sb(Color(0, 0, 0, 0), GOLD1, 2, 6, 18))
    t.set_color("font_color", "Button", GOLD1)
    t.set_color("font_hover_color", "Button", GOLD1)
    t.set_color("font_pressed_color", "Button", GOLD1)
    t.set_color("font_disabled_color", "Button", MUTED)
    for cls in ["LineEdit", "TextEdit"]:
        t.set_stylebox("normal", cls, _sb(Color("#2a2235"), Color("#5a4e68"), 1, 6, 12))
        t.set_stylebox("focus", cls, _sb(Color("#2a2235"), GOLD, 2, 6, 12))
        t.set_color("font_color", cls, PAPER)
        t.set_color("font_placeholder_color", cls, MUTED)
        t.set_color("caret_color", cls, GOLD1)
    return t
func _hard_shadow(l: Label) -> void:
    l.add_theme_color_override("font_shadow_color", Color(0.08, 0.05, 0.1, 0.9))
    l.add_theme_constant_override("shadow_offset_x", 2)
    l.add_theme_constant_override("shadow_offset_y", 2)
func _label(text: String, size := 24, color := PAPER, align := HORIZONTAL_ALIGNMENT_CENTER, wrap :=
true) -> Label:
    var l := Label.new()
    l.text = text
    l.add_theme_font_size_override("font_size", size)
    l.add_theme_color_override("font_color", color)
    l.horizontal_alignment = align
    l.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART if wrap else TextServer.AUTOWRAP_OFF
    return l
func _center_panel(width := 440.0) -> Array:
    var root := Control.new()
    root.set_anchors_preset(Control.PRESET_FULL_RECT)
    root.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var dim := ColorRect.new()
    dim.color = Color(0.06, 0.03, 0.06, 0.55)
    dim.set_anchors_preset(Control.PRESET_FULL_RECT)
    root.add_child(dim)
    var cc := CenterContainer.new()
    cc.set_anchors_preset(Control.PRESET_FULL_RECT)
    root.add_child(cc)
    var p := PanelContainer.new()
    p.custom_minimum_size = Vector2(width, 0)
    cc.add_child(p)
    var box := VBoxContainer.new()
    box.add_theme_constant_override("separation", 14)
```

```
    p.add_child(box)
    ui.add_child(root)
    return [root, box]
func _build_ui() -> void:
    var title := Control.new()
    title.set_anchors_preset(Control.PRESET_FULL_RECT)
    title.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var dim := ColorRect.new()
    dim.color = Color(0.06, 0.02, 0.05, 0.62)
    dim.set_anchors_preset(Control.PRESET_FULL_RECT)
    title.add_child(dim)
    var tv := VBoxContainer.new()
    tv.set_anchors_preset(Control.PRESET_FULL_RECT)
    tv.alignment = BoxContainer.ALIGNMENT_CENTER
    tv.add_theme_constant_override("separation", 40)
    title.add_child(tv)
    var name_l := _label("苏七镇奇缘", 72, GOLD1)
    name_l.add_theme_color_override("font_shadow_color", RED0)
    name_l.add_theme_constant_override("shadow_offset_x", 6)
    name_l.add_theme_constant_override("shadow_offset_y", 6)
    tv.add_child(name_l)
    var open_b := Button.new()
    open_b.text = "打开看看"
    open_b.size_flags_horizontal = Control.SIZE_SHRINK_CENTER
    open_b.pressed.connect(func(): _goto("nick"))
    tv.add_child(open_b)
    ui.add_child(title)
    screens["title"] = title
    var np := _center_panel()
    screens["nick"] = np[0]
    var nb: VBoxContainer = np[1]
    nick_edit = LineEdit.new()
    nick_edit.placeholder_text = "填一个朋友们认得出的名字"
    nick_edit.max_length = 12
    nick_edit.text_changed.connect(func(_t): _check_nick_ui())
    nb.add_child(nick_edit)
    nb.add_child(_label("其他宾客会看到这个名字", 24, MUTED, HORIZONTAL_ALIGNMENT_LEFT))
    var row := HBoxContainer.new()
    nick_err = _label("", 24, Color("#E8746A"), HORIZONTAL_ALIGNMENT_LEFT)
    nick_err.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    nick_count = _label("0/12", 24, MUTED, HORIZONTAL_ALIGNMENT_RIGHT, false)
    row.add_child(nick_err)
    row.add_child(nick_count)
    nb.add_child(row)
    nick_ok = Button.new()
    nick_ok.text = "好"
    nick_ok.disabled = true
    nick_ok.pressed.connect(func():
        nickname = _normalize(nick_edit.text)
        _goto("avatar"))
```

```
    nb.add_child(nick_ok)
    var ap := _center_panel(470.0)
    screens["avatar"] = ap[0]
    var ab: VBoxContainer = ap[1]
    var grid := GridContainer.new()
    grid.columns = 4
    grid.add_theme_constant_override("h_separation", 10)
    grid.add_theme_constant_override("v_separation", 10)
    var group := ButtonGroup.new()
    for i in 8:
        var b := Button.new()
        b.toggle_mode = true
        b.button_group = group
        var gl := Spr.gpt_look(i)
        if gl.is_empty():
            b.icon = Spr.portrait(i, 3)
        else:
            var at := AtlasTexture.new()
            at.atlas = gl["tex"]
            at.region = Rect2(int(gl["anim"]["down"]["idle"]) * int(gl["frame_w"]), 0, gl["frame_w"]
, gl["frame_h"])
            b.icon = at
            b.expand_icon = true
        b.icon_alignment = HORIZONTAL_ALIGNMENT_CENTER
        b.custom_minimum_size = Vector2(96, 124)
        b.add_theme_stylebox_override("normal", _sb(Color("#2a2235"), Color("#5a4e68"), 1, 6, 6))
        b.add_theme_stylebox_override("hover", _sb(Color("#2a2235"), GOLD, 2, 6, 6))
        b.add_theme_stylebox_override("pressed", _sb(Color("#3b2230"), GOLD1, 3, 6, 6))
        b.button_pressed = (i == preset)
        b.pressed.connect(func(): preset = i)
        grid.add_child(b)
    ab.add_child(grid)
    var ok2 := Button.new()
    ok2.text = "好"
    ok2.pressed.connect(_enter_court)
    ab.add_child(ok2)
    var cp := _center_panel()
    screens["clip"] = cp[0]
    var cb: VBoxContainer = cp[1]
    cb.add_child(_label("【待补】两分钟剪辑", 36, GOLD1))
    cb.add_child(_label("六章各一张图一句话，等阿鱼提供。样机先跳过。", 24, MUTED))
    var go := Button.new()
    go.text = "继续"
    go.pressed.connect(_close_clip)
    cb.add_child(go)
    var ep := _center_panel(480.0)
    screens["end"] = ep[0]
    var eb: VBoxContainer = ep[1]
    eb.add_child(_label("【待做】出了罗曼山庄往南", 36, GOLD1))
    eb.add_child(_label("本地人集市、镇政府、小学门口的黄葛树、老街、石板桥", 24, MUTED))
```

```
    var again := Button.new()
    again.text = "再走一遍"
    again.pressed.connect(_enter_court)
    eb.add_child(again)
    _build_hud()
    _build_person_card()
    _build_book_panel()
    fade = ColorRect.new()
    fade.color = Color(0, 0, 0, 0)
    fade.set_anchors_preset(Control.PRESET_FULL_RECT)
    fade.mouse_filter = Control.MOUSE_FILTER_IGNORE
    ui.add_child(fade)
func _build_hud() -> void:
    hud = Control.new()
    hud.set_anchors_preset(Control.PRESET_FULL_RECT)
    hud.mouse_filter = Control.MOUSE_FILTER_IGNORE
    ui.add_child(hud)
    var top_row := HBoxContainer.new()
    top_row.position = Vector2(14, 14)
    top_row.add_theme_constant_override("separation", 10)
    top_row.mouse_filter = Control.MOUSE_FILTER_IGNORE
    hud.add_child(top_row)
    var chip := PanelContainer.new()
    chip.add_theme_stylebox_override("panel", _sb(Color(0.08, 0.06, 0.1, 0.78), Color(GOLD, 0.5), 1,
 14, 10))
    chip.add_child(_label("● 罗曼山庄", 24, GOLD1, HORIZONTAL_ALIGNMENT_LEFT, false))
    top_row.add_child(chip)
    var cchip := PanelContainer.new()
    cchip.name = "coins_chip"
    cchip.add_theme_stylebox_override("panel", _sb(Color(0.08, 0.06, 0.1, 0.78), Color(GOLD, 0.5), 1
, 14, 10))
    coins_chip = _label("0 金", 24, GOLD1, HORIZONTAL_ALIGNMENT_RIGHT, false)
    cchip.add_child(coins_chip)
    top_row.add_child(cchip)
    stick_base = _ring(STICK_R * 2.0, Color(0.08, 0.06, 0.1, 0.32), Color(GOLD1, 0.9))
    stick_knob = _ring(56.0, Color(GOLD1, 0.55), Color(0.08, 0.06, 0.1, 0.8))
    hud.add_child(stick_base)
    hud.add_child(stick_knob)
    _stick_rest.call_deferred()
    hint = PanelContainer.new()
    hint.name = "walk_hint"
    hint.add_theme_stylebox_override("panel", _sb(Color(0.106, 0.078, 0.133, 0.94), GOLD1, 2, 6, 14)
)
    hint.mouse_filter = Control.MOUSE_FILTER_IGNORE
    hint.add_child(_label(LINE_WALK_HINT, 24, GOLD1, HORIZONTAL_ALIGNMENT_LEFT, false))
    hint.set_anchors_preset(Control.PRESET_BOTTOM_RIGHT)
    hint.grow_horizontal = Control.GROW_DIRECTION_BEGIN
    hint.grow_vertical = Control.GROW_DIRECTION_BEGIN
    hint.offset_right = -14
    hint.offset_left = -14
```

```
    hint.offset_top = -262
    hint.offset_bottom = -180
    hint.visible = false
    hud.add_child(hint)
    note_box = PanelContainer.new()
    note_box.set_anchors_preset(Control.PRESET_TOP_WIDE)
    note_box.offset_left = 14; note_box.offset_right = -14
    note_box.add_theme_stylebox_override("panel", _sb(Color(0.106, 0.078, 0.133, 0.94), GOLD1, 2, 6,
14))
    note_box.mouse_filter = Control.MOUSE_FILTER_IGNORE
    note = _label("", 24, GOLD1)
    note_box.add_child(note)
    note_box.visible = false
    hud.add_child(note_box)
    sit_btn = Button.new()
    sit_btn.text = "坐这儿"
    sit_btn.visible = false
    sit_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    sit_btn.offset_top = -170
    sit_btn.offset_bottom = -120
    sit_btn.offset_left = -220
    sit_btn.offset_right = 220
    sit_btn.pressed.connect(_on_sit_pressed)
    hud.add_child(sit_btn)
    person_btn = Button.new()
    person_btn.text = ""
    person_btn.visible = false
    person_btn.clip_text = true
    person_btn.text_overrun_behavior = TextServer.OVERRUN_TRIM_ELLIPSIS
    person_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    person_btn.offset_top = -170
    person_btn.offset_bottom = -120
    person_btn.offset_left = -220
    person_btn.offset_right = 220
    person_btn.pressed.connect(_on_person_pressed)
    hud.add_child(person_btn)
    book_btn = Button.new()
    book_btn.text = "翻簿子"
    book_btn.visible = false
    book_btn.clip_text = true
    book_btn.text_overrun_behavior = TextServer.OVERRUN_TRIM_ELLIPSIS
    book_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    book_btn.offset_top = -170
    book_btn.offset_bottom = -120
    book_btn.pressed.connect(_on_book_btn_pressed)
    hud.add_child(book_btn)
    rel_btn = Button.new()
    rel_btn.visible = false
    rel_btn.clip_text = true
    rel_btn.text_overrun_behavior = TextServer.OVERRUN_TRIM_ELLIPSIS
```

```
    rel_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    rel_btn.offset_top = -170
    rel_btn.offset_bottom = -120
    rel_btn.pressed.connect(func(): _open_rel_card(str(near_rel["title"]), str(near_rel["relation"])
))
    hud.add_child(rel_btn)
    chat_btn = Button.new()
    chat_btn.visible = false
    chat_btn.clip_text = true
    chat_btn.text_overrun_behavior = TextServer.OVERRUN_TRIM_ELLIPSIS
    chat_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    chat_btn.offset_top = -170
    chat_btn.offset_bottom = -120
    chat_btn.pressed.connect(_on_chat_pressed)
    hud.add_child(chat_btn)
    story_btn = Button.new()
    story_btn.text = LINE_STORY_BTN
    story_btn.visible = false
    story_btn.clip_text = true
    story_btn.text_overrun_behavior = TextServer.OVERRUN_TRIM_ELLIPSIS
    story_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    story_btn.offset_top = -170
    story_btn.offset_bottom = -120
    story_btn.pressed.connect(func(): _open_story_card(near_table_id))
    hud.add_child(story_btn)
    play_btn = Button.new()
    play_btn.visible = false
    play_btn.clip_text = true
    play_btn.text_overrun_behavior = TextServer.OVERRUN_TRIM_ELLIPSIS
    play_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    play_btn.offset_top = -170
    play_btn.offset_bottom = -120
    play_btn.pressed.connect(func(): _open_game(str(hall.my_seat.get("table_id", ""))))
    hud.add_child(play_btn)
    add_dish_btn = Button.new()
    add_dish_btn.text = "添一道菜"
    add_dish_btn.visible = false
    add_dish_btn.clip_text = true
    add_dish_btn.text_overrun_behavior = TextServer.OVERRUN_TRIM_ELLIPSIS
    add_dish_btn.set_anchors_preset(Control.PRESET_CENTER_BOTTOM)
    add_dish_btn.offset_top = -170
    add_dish_btn.offset_bottom = -120
    add_dish_btn.pressed.connect(func(): banquet_ui.open_add())
    hud.add_child(add_dish_btn)
    dialog = PanelContainer.new()
    dialog.set_anchors_preset(Control.PRESET_TOP_WIDE)
    dialog.offset_left = 14; dialog.offset_right = -14
    dialog.offset_top = TOP_Y; dialog.offset_bottom = TOP_Y
    dialog.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var dv := VBoxContainer.new()
```

```
    dialog_who = _label("阿鱼", 24, GOLD, HORIZONTAL_ALIGNMENT_LEFT)
    dv.add_child(dialog_who)
    dialog_text = _label("", 24, PAPER, HORIZONTAL_ALIGNMENT_LEFT)
    dv.add_child(dialog_text)
    dialog.add_child(dv)
    dialog.visible = false
    dialog.gui_input.connect(func(e):
        if dialog_gloss != "" and ((e is InputEventScreenTouch or e is InputEventMouseButton) and e.
pressed):
            _note(dialog_gloss, 2.5)
            dialog.accept_event())
    hud.add_child(dialog)
    whisper_box = PanelContainer.new()
    whisper_box.set_anchors_preset(Control.PRESET_BOTTOM_WIDE)
    whisper_box.offset_left = 14; whisper_box.offset_right = -14
    whisper_box.offset_top = -380; whisper_box.offset_bottom = -128
    var wv := VBoxContainer.new()
    wv.add_theme_constant_override("separation", 10)
    whisper_edit = TextEdit.new()
    whisper_edit.custom_minimum_size = Vector2(0, 140)
    whisper_edit.wrap_mode = TextEdit.LINE_WRAPPING_BOUNDARY
    whisper_edit.text_changed.connect(_check_whisper_ui)
    wv.add_child(whisper_edit)
    var wr := HBoxContainer.new()
    whisper_count = _label("0/200", 24, MUTED, HORIZONTAL_ALIGNMENT_LEFT, false)
    whisper_count.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    wr.add_child(whisper_count)
    whisper_ok = Button.new()
    whisper_ok.text = "好"
    whisper_ok.disabled = true
    whisper_ok.pressed.connect(_send_whisper)
    wr.add_child(whisper_ok)
    wv.add_child(wr)
    gift_row = HBoxContainer.new()
    var gl := _label("", 24, GOLD1, HORIZONTAL_ALIGNMENT_LEFT, false)
    gl.name = "coins"
    gl.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    gift_row.add_child(gl)
    var gb := Button.new()
    gb.name = "add"
    gb.text = "添礼金"
    gb.pressed.connect(_open_gift_pick)
    gift_row.add_child(gb)
    wv.add_child(gift_row)
    gift_pick = HBoxContainer.new()
    gift_pick.add_theme_constant_override("separation", 8)
    gift_pick.visible = false
    wv.add_child(gift_pick)
    whisper_box.add_child(wv)
    whisper_box.visible = false
```

```
    hud.add_child(whisper_box)
    toast_back_btn = Button.new()
    toast_back_btn.text = LINE_TOAST_BACK
    toast_back_btn.visible = false
    toast_back_btn.set_anchors_preset(Control.PRESET_TOP_RIGHT)
    toast_back_btn.pressed.connect(_on_toast_back_pressed)
    ui.add_child(toast_back_btn)
func _build_person_card() -> void:
    person_card = Control.new()
    person_card.set_anchors_preset(Control.PRESET_FULL_RECT)
    person_card.visible = false
    var dim := Button.new()
    dim.set_anchors_preset(Control.PRESET_FULL_RECT)
    for st in ["normal", "hover", "pressed", "focus"]:
        dim.add_theme_stylebox_override(st, _sb(Color(0.06, 0.03, 0.06, 0.6), Color(0, 0, 0, 0), 0,
0, 0))
    dim.pressed.connect(_close_person_card)
    person_card.add_child(dim)
    var cc := CenterContainer.new()
    cc.set_anchors_preset(Control.PRESET_FULL_RECT)
    cc.mouse_filter = Control.MOUSE_FILTER_IGNORE
    person_card.add_child(cc)
    var panel := PanelContainer.new()
    panel.custom_minimum_size = Vector2(400.0, 0)
    cc.add_child(panel)
    var box := VBoxContainer.new()
    box.add_theme_constant_override("separation", 12)
    panel.add_child(box)
    if font.has_char(0xD7):
        var head := HBoxContainer.new()
        var spacer := Control.new()
        spacer.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        head.add_child(spacer)
        var close_b := Button.new()
        close_b.text = "×"
        close_b.custom_minimum_size = Vector2(40, 40)
        close_b.pressed.connect(_close_person_card)
        head.add_child(close_b)
        box.add_child(head)
    var pc := CenterContainer.new()
    person_card_portrait = TextureRect.new()
    person_card_portrait.custom_minimum_size = Vector2(140, 220)
    person_card_portrait.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    person_card_portrait.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    pc.add_child(person_card_portrait)
    box.add_child(pc)
    person_card_name = _label("", 36, GOLD1)
    box.add_child(person_card_name)
    person_card_inviter = _label("", 24, MUTED)
    box.add_child(person_card_inviter)
```

```
    var btns := HBoxContainer.new()
    btns.add_theme_constant_override("separation", 12)
    person_card_toast_btn = Button.new()
    person_card_toast_btn.text = "敬一杯"
    person_card_toast_btn.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    person_card_toast_btn.pressed.connect(_on_toast_pressed)
    btns.add_child(person_card_toast_btn)
    person_card_meet_btn = Button.new()
    person_card_meet_btn.text = LINE_MEET
    person_card_meet_btn.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    person_card_meet_btn.pressed.connect(_on_meet_pressed)
    btns.add_child(person_card_meet_btn)
    box.add_child(btns)
    ui.add_child(person_card)
func _set_person_portrait(preset_in: int) -> void:
    var p := preset_in
    if p < 0 or p > 7:
        p = 0
    var gl := Spr.gpt_look(p)
    if gl.is_empty():
        person_card_portrait.texture = Spr.portrait(p, 3)
    else:
        var at := AtlasTexture.new()
        at.atlas = gl["tex"]
        at.region = Rect2(int(gl["anim"]["down"]["idle"]) * int(gl["frame_w"]), 0, gl["frame_w"], gl
["frame_h"])
        person_card_portrait.texture = at
func _build_book_panel() -> void:
    book_panel = Control.new()
    book_panel.set_anchors_preset(Control.PRESET_FULL_RECT)
    book_panel.visible = false
    var dim := Button.new()
    dim.set_anchors_preset(Control.PRESET_FULL_RECT)
    for st in ["normal", "hover", "pressed", "focus"]:
        dim.add_theme_stylebox_override(st, _sb(Color(0.06, 0.03, 0.06, 0.6), Color(0, 0, 0, 0), 0,
0, 0))
    dim.pressed.connect(_close_book_panel)
    book_panel.add_child(dim)
    var cc := CenterContainer.new()
    cc.set_anchors_preset(Control.PRESET_FULL_RECT)
    cc.mouse_filter = Control.MOUSE_FILTER_IGNORE
    book_panel.add_child(cc)
    var panel := PanelContainer.new()
    panel.custom_minimum_size = Vector2(460.0, 0)
    cc.add_child(panel)
    var box := VBoxContainer.new()
    box.add_theme_constant_override("separation", 12)
    panel.add_child(box)
    if font.has_char(0xD7):
        var head := HBoxContainer.new()
```

```
        var spacer := Control.new()
        spacer.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        head.add_child(spacer)
        var close_b := Button.new()
        close_b.text = "×"
        close_b.custom_minimum_size = Vector2(40, 40)
        close_b.pressed.connect(_close_book_panel)
        head.add_child(close_b)
        box.add_child(head)
    book_scroll = ScrollContainer.new()
    book_scroll.custom_minimum_size = Vector2(0, 220)
    book_scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    book_list_box = VBoxContainer.new()
    book_list_box.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    book_list_box.add_theme_constant_override("separation", 6)
    book_scroll.add_child(book_list_box)
    box.add_child(book_scroll)
    book_loading = _label("加载中", 24, MUTED, HORIZONTAL_ALIGNMENT_LEFT)
    box.add_child(book_loading)
    book_readonly_label = _label(LINE_BOOK_READONLY, 24, MUTED, HORIZONTAL_ALIGNMENT_LEFT)
    book_readonly_label.visible = false
    box.add_child(book_readonly_label)
    book_input_row = VBoxContainer.new()
    book_input_row.add_theme_constant_override("separation", 10)
    book_edit = TextEdit.new()
    book_edit.custom_minimum_size = Vector2(0, 80)
    book_edit.wrap_mode = TextEdit.LINE_WRAPPING_BOUNDARY
    book_edit.placeholder_text = BOOK_PLACEHOLDER
    book_edit.text_changed.connect(_check_book_ui)
    book_input_row.add_child(book_edit)
    var br := HBoxContainer.new()
    book_err = _label("", 24, Color("#E8746A"), HORIZONTAL_ALIGNMENT_LEFT)
    book_err.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    book_count = _label("0/100", 24, MUTED, HORIZONTAL_ALIGNMENT_RIGHT, false)
    br.add_child(book_err)
    br.add_child(book_count)
    book_input_row.add_child(br)
    book_ok = Button.new()
    book_ok.text = "好"
    book_ok.disabled = true
    book_ok.pressed.connect(_on_book_ok_pressed)
    book_input_row.add_child(book_ok)
    box.add_child(book_input_row)
    ui.add_child(book_panel)
func _open_person_card(openid: String) -> void:
    if openid == "" or whisper_box.visible or book_open:
        return
    card_open = true
    card_openid = openid
    var info: Dictionary = hall.resident_names.get(openid, {})
```

```
    var nick := str(info.get("nick", ""))
    person_card_name.text = nick if nick != "" else "有人"
    var inviter_raw = info.get("inviter", "")
    var inviter := "" if inviter_raw == null else str(inviter_raw)
    var inviter_nick := ""
    if inviter != "":
        inviter_nick = str((hall.resident_names.get(inviter, {}) as Dictionary).get("nick", ""))
    if inviter != "" and inviter_nick != "":
        person_card_inviter.text = inviter_nick + " 带进来的"
        person_card_inviter.visible = true
    else:
        person_card_inviter.text = ""
        person_card_inviter.visible = false
    _set_person_portrait(int(info.get("avatar", -1)))
    person_card_toast_btn.disabled = false
    person_card_toast_btn.text = "敬一杯"
    person_card_meet_btn.disabled = false
    person_card_meet_btn.text = LINE_MEET
    person_card.visible = true
    hall.walking_enabled = false
    _refresh_seat_buttons()
func _close_person_card() -> void:
    if not card_open:
        return
    card_open = false
    card_openid = ""
    person_card.visible = false
    if state == "banquet" and not whisper_box.visible and not book_open:
        hall.walking_enabled = true
    _refresh_seat_buttons()
func _in_scene() -> bool:
    return state in ["court", "banquet", "backhall", "lawn", "lake", "orchard"]
func _show_canvas(which: String) -> void:
    var canvases := {"court": court, "banquet": hall, "backhall": backhall, "lawn": lawn, "lake": la
ke, "orchard": orchard}
    for k in canvases:
        var c = canvases[k]
        if c == null:
            continue
        var on: bool = k == which
        c.visible = on
        c.process_mode = Node.PROCESS_MODE_INHERIT if on else Node.PROCESS_MODE_DISABLED
    cv = canvases[which] if canvases.get(which) != null else hall
    if cv.cam != null:
        cv.cam.make_current()
func _goto(s: String) -> void:
    state = s
    hall.stick = Vector2.ZERO
    court.stick = Vector2.ZERO
    if backhall != null:
```

```
        backhall.stick = Vector2.ZERO
    if lawn != null:
        lawn.stick = Vector2.ZERO
    if lake != null:
        lake.stick = Vector2.ZERO
    if orchard != null:
        orchard.stick = Vector2.ZERO
    stick_touch = -1
    touches.clear()
    for k in screens:
        screens[k].visible = (k == s)
    hud.visible = s in ["banquet", "court", "backhall", "lawn", "lake", "orchard"]
    if sit_btn != null and s != "banquet":
        sit_btn.visible = false
    if person_btn != null and s != "banquet":
        person_btn.visible = false
    if chat_btn != null and s != "banquet":
        chat_btn.visible = false
    if rel_btn != null and s != "banquet":
        rel_btn.visible = false
    if book_btn != null and s != "banquet":
        book_btn.visible = false
    if story_btn != null and s != "banquet":
        story_btn.visible = false
        _close_story_card()
    if play_btn != null and s != "banquet":
        play_btn.visible = false
    if add_dish_btn != null and s != "banquet":
        add_dish_btn.visible = false
    if banquet_ui != null and s != "banquet":
        banquet_ui.hide_all()
        var life = _hall_life()
        if life != null:
            life.clear_orders()
    if is_instance_valid(game_host) and s != "banquet":
        game_host.queue_free()
        game_host = null
    if person_card != null and s != "banquet" and card_open:
        card_open = false
        card_openid = ""
        person_card.visible = false
    if book_panel != null and s != "banquet" and book_open:
        book_open = false
        book_table = ""
        book_panel.visible = false
    hall.walking_enabled = (s == "banquet") and not whisper_box.visible and not _menu_open()
    court.walking_enabled = (s == "court") and not court_ui.panel_open()
    if s != "court":
        court_ui.hide_all()
    if backhall != null:
```

```
        backhall.walking_enabled = (s == "backhall")
    if backhall_ui != null and s != "backhall":
        backhall_ui.hide_all()
    if lawn != null:
        lawn.walking_enabled = (s == "lawn")
    if lawn_ui != null and s != "lawn":
        lawn_ui.hide_all()
    if lake != null:
        lake.walking_enabled = (s == "lake")
    if lake_ui != null and s != "lake":
        lake_ui.hide_all()
    if orchard != null:
        orchard.walking_enabled = (s == "orchard")
    if orchard_ui != null and s != "orchard":
        orchard_ui.hide_all()
    if s == "nick":
        nick_edit.grab_focus()
        _check_nick_ui()
func _fade_to(a: float) -> void:
    if fast:
        fade.color.a = a
        return
    var tw := create_tween()
    tw.tween_property(fade, "color:a", a, 0.35)
    await tw.finished
func _enter_court() -> void:
    await _fade_to(1.0)
    hall.set_look(preset, nickname)
    court.set_look(preset, nickname)
    backhall.set_look(preset, nickname)
    lawn.set_look(preset, nickname)
    lake.set_look(preset, nickname)
    orchard.set_look(preset, nickname)
    hall.couple_armed = true
    hall.exit_armed = false
    hall.exit_mark.visible = false
    whisper_box.visible = false
    hall.reset_to(hall.SPAWN)
    hall_side_latched = true
    court.reset_to(court.SPAWN)
    _show_canvas("court")
    _goto("court")
    photo_greeted = false
    rest_heard = false
    _update_gate()
    hall_ready = false
    hint.visible = not hint_done
    await _fade_to(0.0)
    _say(LINE_ARRIVE)
    _save_profile()
```

```
func _save_profile() -> void:
    var inv: String = cloud.inviter()
    var payload := profile_payload(nickname, preset, inv, cloud.openid())
    print("[suqi] 存资料 带邀请人 ", payload.has("inviter"), " ", inv.substr(0, 6))
    cloud.call_fn("saveProfile", payload, _after_profile)
func _switch_canvas(to: String) -> void:
    if switching or not _in_scene():
        return
    switching = true
    var from := state
    var keep_face: String = cv.face
    cv.walking_enabled = false
    cv.stick = Vector2.ZERO
    cv.path.clear()
    fade.color = Color(SWITCH_TINT, fade.color.a)
    await _fade_to(1.0)
    if from == "lawn":
        lawn.reset_ball()
    if to == "banquet":
        _show_canvas("banquet")
        var from_back: Vector2 = hall.side_door + Vector2(-60, 0) if USE_LAYERED and hall.big else H
ALL_FROM_BACKHALL
        hall.reset_to(from_back if from == "backhall" else hall.SPAWN)
        hall_door_latched = true
        hall_side_latched = true
        _goto("banquet")
        _enter_hall_net()
    elif to == "backhall":
        _show_canvas("backhall")
        backhall.reset_to(BACKHALL_FROM_LAWN if from == "lawn" else BACKHALL_FROM_HALL)
        _goto("backhall")
    elif to == "lawn":
        _show_canvas("lawn")
        lawn.reset_ball()
        var lawn_at := LAWN_FROM_COURT
        if from == "backhall":
            lawn_at = LAWN_FROM_BACKHALL
        elif from == "orchard":
            lawn_at = LAWN_FROM_ORCHARD
        lawn.reset_to(lawn_at)
        kids_heard = false
        _goto("lawn")
    elif to == "lake":
        _show_canvas("lake")
        lake.reset_to(LAKE_FROM_COURT)
        _goto("lake")
    elif to == "orchard":
        _show_canvas("orchard")
        orchard.reset_to(ORCHARD_FROM_LAWN)
        orchard.refresh_fruit()
```

```
        _goto("orchard")
    else:
        _show_canvas("court")
        var back_to := COURT_FROM_HALL
        if from == "lawn":
            back_to = COURT_FROM_LAWN
        elif from == "lake":
            back_to = COURT_FROM_LAKE
        court.reset_to(back_to)
        _goto("court")
        photo_greeted = false
        rest_heard = false
        _update_gate()
    cv.face = keep_face
    print("[suqi] 换画布 到 ", to)
    await _fade_to(0.0)
    fade.color = Color(0, 0, 0, 0)
    switching = false
func _enter_hall_net() -> void:
    hall_ready = false
    pulse_since = 0
    pulse_t = 0.0
    pulse_seen.clear()
    if profile_saved:
        cloud.call_fn("enterHall", {}, _on_enter_hall)
    else:
        _save_profile()
func _on_court_spot(id: String, inside: bool) -> void:
    if state != "court" or switching:
        return
    court_ui.spot(id, inside)
    match id:
        "welcome":
            if inside and not court_save.welcomed:
                court_save.welcomed = true
                court_save.save()
                if not court_save.signed_in:
                    _say(CourtCopy.text("welcome"), -1.0, CourtCopy.who("welcome"))
        "photo_wait":
            if inside and not photo_greeted:
                photo_greeted = true
                _say(CourtCopy.text("photo_greet"), -1.0, CourtCopy.who("photo_greet"))
        "rest":
            if inside and not rest_heard:
                rest_heard = true
                _say(CourtCopy.text("elder"), -1.0, CourtCopy.who("elder"), CourtCopy.text("elder_gl
oss"))
                if not court_save.gloss_seen:
                    court_save.gloss_seen = true
                    court_save.save()
```

```
                    _note(CourtCopy.text("elder_gloss"), 3.0)
        "gate":
            if inside:
                _at_gate()
        "to_hall":
            if inside:
                _switch_canvas("banquet")
        "to_lawn":
            if inside:
                _switch_canvas("lawn")
        "to_lake":
            if inside:
                _switch_canvas("lake")
func _on_backhall_spot(id: String, inside: bool) -> void:
    if state != "backhall" or switching:
        return
    backhall_ui.spot(id, inside)
    match id:
        "to_hall":
            if inside:
                _switch_canvas("banquet")
        "to_lawn":
            if inside:
                _switch_canvas("lawn")
func _on_lawn_spot(id: String, inside: bool) -> void:
    if state != "lawn" or switching:
        return
    lawn_ui.spot(id, inside)
    match id:
        "to_entrance":
            if inside:
                _switch_canvas("court")
        "to_backhall":
            if inside:
                _switch_canvas("backhall")
        "to_orchard":
            if inside:
                _switch_canvas("orchard")
func _on_orchard_spot(id: String, inside: bool) -> void:
    if state != "orchard" or switching:
        return
    orchard_ui.spot(id, inside)
    if id == "to_lawn" and inside:
        _switch_canvas("lawn")
func _on_lake_spot(id: String, inside: bool) -> void:
    if state != "lake" or switching:
        return
    lake_ui.spot(id, inside)
    if id == "to_entrance" and inside:
        _switch_canvas("court")
```

```
                filePath: res.tempFilePath,
                success: function (r) {
                    var t2 = Date.now();
                    var buf = r && r.data;
                    if (!buf || typeof buf.byteLength !== 'number') {
                        done({ ok: false, reason: 'read_failed', detail: '读回来的不是 ArrayBuffer'
});
                        return;
                    }
                    try {
                        sdk.copy_to_fs(d.path, buf);
                    } catch (e) {
                        done({ ok: false, reason: 'copy_failed', detail: String(e && e.message ? e.m
essage : e) });
                        return;
                    }
                    var t3 = Date.now();
                    console.log('[suqi] 远程加载 写进 ' + d.path + ' ' + buf.byteLength + ' 字节');
                    done({ ok: true, bytes: buf.byteLength, download_ms: t1 - t0, read_ms: t2 - t1,
write_ms: t3 - t2 });
                },
                fail: function (err) {
                    done({ ok: false, reason: 'read_failed', detail: err && err.errMsg ? err.errMsg
: String(err) });
                },
            });
        },
        fail: function (err) {
            var ms = Date.now() - t0;
            diagnose(d.fileID, function (diag) {
                done({ ok: false, reason: 'download_failed', detail: err && err.errMsg ? err.errMsg
: String(err), download_ms: ms, diag: diag });
            });
        },
    });
};
var B64 = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/';
var B64_MAP = null;
function b64Into(s, out, at) {
    if (!B64_MAP) {
        B64_MAP = new Int16Array(128);
        for (var i = 0; i < 128; i++) B64_MAP[i] = -1;
        for (var j = 0; j < 64; j++) B64_MAP[B64.charCodeAt(j)] = j;
    }
    var n = 0;
    var bits = 0;
    var acc = 0;
    for (var k = 0; k < s.length; k++) {
        var c = s.charCodeAt(k);
        if (c === 61) break;
```

```
        var v = c < 128 ? B64_MAP[c] : -1;
        if (v < 0) continue;
        acc = ((acc << 6) | v) & 0xffffff;
        bits += 6;
        if (bits >= 8) {
            bits -= 8;
            if (at + n >= out.length) return -1;
            out[at + n] = (acc >> bits) & 0xff;
            n++;
        }
    }
    return n;
}
GameGlobal.__suqiB64Into = b64Into;
GameGlobal.__suqiLocalFns['@fetchChunks'] = function (d, done) {
    var t0 = Date.now();
    if (!d || !d.fileID || !d.path) {
        done({ ok: false, reason: 'bad_request' });
        return;
    }
    var sdk = GameGlobal.GODOTSDK;
    if (!sdk || typeof sdk.copy_to_fs !== 'function') {
        done({ ok: false, reason: 'no_copy_to_fs', detail: '模板里没有 GODOTSDK.copy_to_fs' });
        return;
    }
    if (typeof wx === 'undefined' || !wx.cloud || !wx.cloud.callFunction) {
        done({ ok: false, reason: 'network', detail: '没有 wx.cloud.callFunction' });
        return;
    }
    var buf = null;
    var got = 0;
    var chunks = 0;
    var retried = 0;
    console.log('[suqi] 远程加载 走云函数 canvasFile ' + String(d.fileID).slice(-60));
    function fail(reason, detail) {
        console.log('[suqi] 远程加载 云函数这路失败 ' + reason + ' ' + detail);
        done({ ok: false, reason: reason, detail: detail, download_ms: Date.now() - t0, chunks: chun
ks });
    }
    function ask(offset, tries) {
        wx.cloud.callFunction({
            name: 'canvasFile',
            data: { fileID: d.fileID, offset: offset },
            success: function (res) {
                var r = res && res.result;
                if (!r || !r.ok) {
                    fail((r && r.reason) || 'download_failed', 'canvasFile 第 ' + (chunks + 1) + '
块: ' + ((r && (r.detail || r.reason)) || '没回结果'));
                    return;
                }
```

```
                if (!buf) {
                    if (!(r.total > 0)) {
                        fail('download_failed', 'canvasFile 回的文件大小不对 ' + r.total);
                        return;
                    }
                    buf = new Uint8Array(r.total);
                }
                if (r.offset !== got || r.total !== buf.length) {
                    fail('download_failed', 'canvasFile 回的位置对不上 ' + r.offset + '/' + got);
                    return;
                }
                var n = b64Into(String(r.data || ''), buf, got);
                if (n !== r.size) {
                    fail('download_failed', '第 ' + (chunks + 1) + ' 块解出来 ' + n + ' 字节，应是 '
 + r.size);
                    return;
                }
                got += n;
                chunks++;
                if (r.done || got >= buf.length) {
                    finish();
                    return;
                }
                ask(got, 0);
            },
            fail: function (err) {
                var msg = err && err.errMsg ? err.errMsg : String(err);
                if (/FunctionName|-501000|not found|不存在/i.test(msg)) {
                    fail('no_function', '云函数 canvasFile 还没部署：' + msg);
                    return;
                }
                if (tries < 1) {
                    retried++;
                    ask(offset, tries + 1);
                    return;
                }
                fail('download_failed', 'canvasFile 第 ' + (chunks + 1) + ' 块：' + msg);
            },
        });
    }
    function finish() {
        var t1 = Date.now();
        try {
            sdk.copy_to_fs(d.path, buf.buffer);
        } catch (e) {
            fail('copy_failed', String(e && e.message ? e.message : e));
            return;
        }
        var t2 = Date.now();
        console.log('[suqi] 远程加载 云函数 ' + chunks + ' 块 ' + got + ' 字节 写进 ' + d.path);
```

```
        done({ ok: true, bytes: got, download_ms: t1 - t0, read_ms: 0, write_ms: t2 - t1, chunks: ch
unks, retried: retried, via: 'function' });
    }
    ask(0, 0);
};
function myOpenid() {
    return (GameGlobal.__suqiUser && GameGlobal.__suqiUser.openid) || '';
}
function card() {
    var me = myOpenid();
    return {
        title: '苏七镇奇缘',
        imageUrl: 'images/share.jpg',
        query: me ? 'from=' + me : '',
    };
}
function short(id) {
    return id ? String(id).slice(0, 6) + '…' : ' (空) ';
}
function takeInviter(opts, when) {
    var q = (opts && opts.query) || {};
    var from = q.from ? String(q.from) : '';
    if (from && from.charAt(0) !== 'L' && !GameGlobal.__suqiInviter) {
        GameGlobal.__suqiInviter = from;
    }
    console.log('[suqi] ' + when + ' 场景 ' + (opts && opts.scene) +
        (from ? ' 卡片上的邀请人 ' + short(from) : ' 没有邀请人') +
        ' 记下的邀请人 ' + short(GameGlobal.__suqiInviter || ''));
}
GameGlobal.__suqiInviter = GameGlobal.__suqiInviter || '';
if (typeof wx !== 'undefined') {
    if (wx.showShareMenu) wx.showShareMenu({ withShareTicket: true, menus: ['shareAppMessage', 'shar
eTimeline'] });
    if (wx.onShareAppMessage) wx.onShareAppMessage(card);
    if (wx.onShareTimeline) {
        wx.onShareTimeline(function () {
            var c = card();
            return { title: c.title, imageUrl: c.imageUrl, query: c.query };
        });
    }
    if (wx.getLaunchOptionsSync) takeInviter(wx.getLaunchOptionsSync(), '启动');
    if (wx.onShow) wx.onShow(function (res) { takeInviter(res, '回到前台'); });
}
'use strict';
const MAX_PAGE_SIZE = 100;
async function fetchAllPages(queryFactory, opts) {
  if (typeof queryFactory !== 'function') {
    throw new Error('fetchAllPages: queryFactory 必须是函数，每次返回一个新查询');
  }
  const maxTotal = opts && opts.maxTotal;
```

```
  if (!(maxTotal > 0)) {
    throw new Error('fetchAllPages: maxTotal 必须是一个正数（防止分页失控），业务代码从 config.js 取
');
  }
  const orderBy = (opts && opts.orderBy) || [['_id', 'asc']];
  const pageSize = Math.min((opts && opts.pageSize) || MAX_PAGE_SIZE, MAX_PAGE_SIZE);
  const all = [];
  let skip = 0;
  for (;;) {
    const remaining = maxTotal - all.length;
    if (remaining <= 0) break;
    const take = Math.min(pageSize, remaining);
    let q = queryFactory();
    for (const [field, dir] of orderBy) q = q.orderBy(field, dir);
    const { data } = await q.skip(skip).limit(take).get();
    all.push(...data);
    if (data.length < take) break;
    skip += data.length;
  }
  return all;
}
module.exports = { fetchAllPages, MAX_PAGE_SIZE };
'use strict';
const C = require('./config');
function assignSeat(ctx) {
  const main = ctx.mainTable;
  if (main && ctx.me && ctx.vip && ctx.vip.has(ctx.me) && main.seats.length < C.BANQUET_MAIN_VIP_CHA
IRS.length) {
    return { ok: true, tableId: main.id, bench: false };
  }
  const onlyWhenFull = ctx.benchOnlyWhenFull === undefined ? C.BANQUET_BENCH_ONLY_WHEN_FULL : !!ctx.
benchOnlyWhenFull;
  return onlyWhenFull ? assignSeatFillFirst(ctx) : assignSeatOld(ctx);
}
function mainTableIdOf(tableIds) {
  const ids55 = C.BANQUET_TABLE_IDS_55;
  if (!Array.isArray(tableIds) || tableIds.length !== ids55.length) return null;
  return tableIds.every((id, i) => id === ids55[i]) ? C.BANQUET_MAIN_TABLE_ID : null;
}
function assignSeatOld(ctx) {
  const { inviter, vip, tables } = ctx;
  const CAP = C.BANQUET_SEATS_PER_TABLE;
  if (!inviter) {
    const t = tables.find(t => t.seats.length < CAP);
    return t ? { ok: true, tableId: t.id, bench: false } : { ok: true, bench: true };
  }
  const inviterTable = tables.find(t => t.seats.includes(inviter));
  if (inviterTable && inviterTable.seats.length < CAP) {
    return { ok: true, tableId: inviterTable.id, bench: false };
  }
```

```
  const branchTables = tables.filter(t => t.branch === inviter);
  const withRoom = branchTables.find(t => t.seats.length < CAP);
  if (withRoom) return { ok: true, tableId: withRoom.id, bench: false };
  if (vip && vip.has(inviter)) {
    const reserved = tables.find(t => t.reserved && t.branch == null && t.seats.length < CAP);
    if (reserved) return { ok: true, tableId: reserved.id, bench: false };
  }
  const empty = tables.find(t => !t.reserved && t.seats.length === 0);
  if (empty) return { ok: true, tableId: empty.id, bench: false };
  return { ok: true, bench: true };
}
function assignSeatFillFirst(ctx) {
  const { inviter, vip, tables, me } = ctx;
  const CAP = C.BANQUET_SEATS_PER_TABLE;
  const hasRoom = t => t.seats.length < CAP;
  const seat = t => ({ ok: true, tableId: t.id, bench: false });
  const vipBranch = !!(vip && (inviter ? vip.has(inviter) : (me && vip.has(me))));
  if (inviter) {
    const inviterTable = tables.find(t => t.seats.includes(inviter));
    if (inviterTable && hasRoom(inviterTable)) return seat(inviterTable);
    const branchTable = tables.find(t => t.branch === inviter && hasRoom(t));
    if (branchTable) return seat(branchTable);
  }
  if (vipBranch) {
    const reserved = tables.find(t => t.reserved && t.branch == null && hasRoom(t));
    if (reserved) return seat(reserved);
  }
  if (inviter) {
    const empty = tables.find(t => !t.reserved && t.seats.length === 0);
    if (empty) return seat(empty);
  }
  const normal = tables.find(t => !t.reserved && hasRoom(t));
  if (normal) return seat(normal);
  if (vipBranch || !inviter) {
    const reserved = tables.find(t => t.reserved && hasRoom(t));
    if (reserved) return seat(reserved);
  }
  return { ok: true, bench: true };
}
function checkSeatTarget(ctx) {
  const { tableId, chairNo } = ctx;
  const ids = ctx.tableIds || C.BANQUET_TABLE_IDS;
  const isMain = !!ctx.mainTableId && tableId === ctx.mainTableId;
  if (typeof tableId !== 'string' || !(isMain || ids.includes(tableId))) return { ok: false, reason:
 'bad_request' };
  if (!Number.isInteger(chairNo) || chairNo < 1 || chairNo > C.BANQUET_SEATS_PER_TABLE) return { ok:
 false, reason: 'bad_request' };
  if (isMain && !ctx.isVip) return { ok: false, reason: 'main_table_only_family' };
  if (isMain && !C.BANQUET_MAIN_VIP_CHAIRS.includes(chairNo)) return { ok: false, reason: 'seat_take
n' };
```

```
  return { ok: true };
}
function canChangeSeat(ctx) {
  const { occupant } = ctx;
  if (occupant) return { ok: false, reason: 'seat_taken' };
  return { ok: true };
}
function canToast(ctx) {
  const { from, to, todayToastCount, todayToastToCount, testMode } = ctx;
  if (from === to) return { ok: false, reason: 'toast_self' };
  if (!testMode && todayToastCount >= C.BANQUET_TOAST_DAILY) return { ok: false, reason: 'toast_limi
t' };
  if (!testMode && todayToastToCount >= C.BANQUET_TOAST_PER_PERSON_DAILY) return { ok: false, reason
: 'toast_same_person' };
  return { ok: true };
}
function canWantToMeet(ctx) {
  const { from, to, alreadySent, fromTableId, toTableId, toasted } = ctx;
  if (from === to) return { ok: false, reason: 'meet_self' };
  if (alreadySent) return { ok: false, reason: 'meet_already' };
  if (fromTableId && fromTableId === toTableId && fromTableId !== C.BANQUET_BENCH_ID) return { ok: t
rue, via: 'table' };
  if (toasted) return { ok: true, via: 'toast' };
  return { ok: false, reason: 'meet_not_met' };
}
function canWriteBook(ctx) {
  const { table, mySeatTableId, text } = ctx;
  if (mySeatTableId !== table) return { ok: false, reason: 'not_sat_here' };
  if (!text) return { ok: false, reason: 'bad_request' };
  if (text.length > C.BANQUET_BOOK_MAX_LEN) return { ok: false, reason: 'book_too_long' };
  return { ok: true };
}
function arrivalDigest(ctx) {
  const { toasts, bookLines, since } = ctx;
  const newToasts = (toasts || []).filter(t => t.at > since);
  const newBookLines = (bookLines || []).filter(l => l.at > since);
  return {
    toasts: newToasts,
    toastCount: newToasts.length,
    bookLineCount: newBookLines.length,
    hasNews: newToasts.length > 0 || newBookLines.length > 0,
  };
}
function arrivalCard(ctx) {
  const { me, since } = ctx;
  const ROWS = C.BANQUET_ARRIVAL_ROWS;
  const seen = new Set([me]);
  const items = [];
  function collect(kind, list, whoKey) {
    const byWho = new Map();
```

```
    for (const x of list || []) {
      const who = x[whoKey];
      if (!who || !(x.at > since)) continue;
      const cur = byWho.get(who);
      if (cur) { cur.count++; cur.at = Math.max(cur.at, x.at); } else byWho.set(who, { kind, who, at
: x.at, count: 1 });
    }
    const group = Array.from(byWho.values()).filter(it => !seen.has(it.who));
    group.sort((a, b) => b.at - a.at);
    for (const it of group) { seen.add(it.who); items.push(it); }
  }
  collect('toast', ctx.toasts, 'from');
  collect('book', ctx.bookLines, 'from');
  collect('visit', ctx.visitors, 'openid');
  for (const it of items) if (it.kind === 'visit') it.count = 1;
  if (items.length <= ROWS) return { rows: items, hasNews: items.length > 0 };
  const rows = items.slice(0, ROWS - 1);
  rows.push({ kind: 'more', count: items.length - (ROWS - 1) });
  return { rows, hasNews: true };
}
module.exports = { assignSeat, mainTableIdOf, checkSeatTarget, canChangeSeat, canToast, canWantToMee
t, canWriteBook, arrivalDigest, arrivalCard };
'use strict';
const C = require('./config');
const HOST_RE = /^[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)+$/;
const SEG_RE = /^[A-Za-z0-9_.-]+$/;
const MAX_LEN = 512;
function canReadCanvas(fileID) {
  if (typeof fileID !== 'string') return false;
  if (fileID.length === 0 || fileID.length > MAX_LEN) return false;
  const PREFIX = 'cloud://';
  if (!fileID.startsWith(PREFIX)) return false;
  const rest = fileID.slice(PREFIX.length);
  const slash = rest.indexOf('/');
  if (slash <= 0) return false;
  if (!HOST_RE.test(rest.slice(0, slash))) return false;
  const path = rest.slice(slash + 1);
  if (!path.startsWith('canvas/')) return false;
  if (path.includes('..')) return false;
  const segs = path.split('/');
  if (segs.length < 2) return false;
  for (const seg of segs) {
    if (seg === '' || seg === '.') return false;
    if (!SEG_RE.test(seg)) return false;
  }
  return true;
}
function chunkRange(total, offset, chunk = C.CANVAS_CHUNK) {
  if (!Number.isInteger(offset) || offset < 0 || offset > total) return { ok: false, reason: 'bad_re
quest' };
```

```
  const end = Math.min(offset + chunk, total);
  return { ok: true, start: offset, end, done: end >= total };
}
module.exports = { canReadCanvas, chunkRange };
'use strict';
const HOUR = 3600 * 1000;
const DAY = 24 * HOUR;
const tableIds = (n) => Array.from({ length: n }, (_, i) => `T${i + 1}`);
module.exports = {
  HOUR,
  DAY,
  TZ_OFFSET: 8 * HOUR,
  CROPS: {
    loubo:   { label: '楼卜', pinyin: '萝卜', growMs: 4 * HOUR, seedPrice: 2,  basePrice: 5  },
    haijiao: { label: '海椒', pinyin: '辣椒', growMs: 1 * DAY,  seedPrice: 5,  basePrice: 15 },
    guzi:    { label: '谷子', pinyin: '稻米', growMs: 3 * DAY,  seedPrice: 15, basePrice: 50 },
  },
  YIELD_PER_PLOT: 10,
  ROT_AFTER_MS: 24 * HOUR,
  PLOT_COUNT: 6,
  FREE_SEEDS: { loubo: 3 },
  START_COIN: 20,
  STEAL_RATIO: 0.3,
  STEAL_DAILY_LIMIT: 10,
  STEAL_PER_HOUSE_LIMIT: 3,
  GUARD_BLOCK_RATE: 0.5,
  GUARD_DURATION_MS: 24 * HOUR,
  GUARD_DAILY_BORROW: 1,
  TOURIST_BUY_MULT: 2.5,
  TOURIST_SEED_MULT: 2,
  LAMP_COST: 30,
  LAMP_TOTAL: 17,
  LAMP_NAME_TOP: 5,
  INVITE_QUOTA_BASE: 5,
  INVITE_QUOTA_PER_WEEK: 1,
  SUGGEST_LIMIT: 10,
  THREAD_WEEKLY_QUOTA: 2,
  THREAD_EXPIRE_MS: 7 * DAY,
  STICK_DAILY: 1,
  STICK_RETURN_COOLDOWN_MS: 7 * DAY,
  BALL_COUNT: 4,
  BALL_GOAL: 10,
  BALL_REWARD_COIN: 5,
  LEN: {
    nickname: [2, 12],
    checkInName: [1, 12],
    message: [1, 100],
    whisper: [1, 200],
    status: [1, 100],
    threadNote: [1, 30],
```

```
    recommend: [1, 50],
  },
  HOUSE_CAPACITY: 5,
  RENAME_LIMIT: 1,
  BANQUET_TABLE_COUNT: 55,
  BANQUET_SEATS_PER_TABLE: 10,
  BANQUET_RESERVED_TABLES: 2,
  BANQUET_TABLE_IDS: tableIds(54),
  BANQUET_TABLE_IDS_55: tableIds(54),
  BANQUET_MAIN_TABLE_ID: 'MAIN',
  BANQUET_MAIN_VIP_CHAIRS: [1, 2],
  BANQUET_BENCH_ONLY_WHEN_FULL: true,
  BANQUET_BENCH_ID: 'bench',
  BANQUET_TOAST_DAILY: 5,
  BANQUET_TOAST_PER_PERSON_DAILY: 1,
  BANQUET_BOOK_MAX_LEN: 100,
  BANQUET_ARRIVAL_ROWS: 3,
  BANQUET_ARRIVAL_LIST_MAX: 30,
  BANQUET_PAGE_MAX_TOTAL: 5000,
  BANQUET_BOOK_READ_MAX: 200,
  BANQUET_TEST_MODE: true,
  CANVAS_CHUNK: 256 * 1024,
};
'use strict';
const C = require('./config');
const T = require('./time');
function plotState(plot, now) {
  if (!plot || !plot.crop) return { state: 'empty', crop: null, remainMs: 0, ripeAt: null, rotAt: nu
ll };
  const cfg = C.CROPS[plot.crop];
  if (!cfg) throw new Error(`未知作物: ${plot.crop}`);
  const ripeAt = plot.plantedAt + cfg.growMs;
  const rotAt = ripeAt + C.ROT_AFTER_MS;
  if (now < ripeAt) return { state: 'growing', crop: plot.crop, remainMs: ripeAt - now, ripeAt, rotA
t };
  if (now < rotAt)  return { state: 'ripe',    crop: plot.crop, remainMs: rotAt - now,  ripeAt, rotA
t };
  return { state: 'rotten', crop: plot.crop, remainMs: 0, ripeAt, rotAt };
}
function canPlant(plot, crop, seedBag, now) {
  if (!C.CROPS[crop]) return { ok: false, reason: 'unknown_crop' };
  if (plotState(plot, now).state !== 'empty') return { ok: false, reason: 'plot_occupied' };
  if ((seedBag[crop] || 0) < 1) return { ok: false, reason: 'no_seed' };
  return { ok: true };
}
function harvestYield(plot, now) {
  if (plotState(plot, now).state !== 'ripe') return 0;
  const stolen = plot.stolenCount || 0;
  const left = C.YIELD_PER_PLOT * (1 - C.STEAL_RATIO * stolen);
  return Math.max(0, Math.floor(left));
```

```
}
function stealYield() {
  return Math.floor(C.YIELD_PER_PLOT * C.STEAL_RATIO);
}
function sellPrice(crop, count, market, now) {
  const cfg = C.CROPS[crop];
  if (!cfg) throw new Error(`未知作物: ${crop}`);
  if (market === 'local' && !T.isMarketDay(now)) return { ok: false, reason: 'local_market_closed' }
;
  const mult = market === 'tourist' ? C.TOURIST_BUY_MULT : 1;
  return { ok: true, coin: Math.floor(cfg.basePrice * count * mult) };
}
function seedPrice(crop, count, market, now) {
  const cfg = C.CROPS[crop];
  if (!cfg) throw new Error(`未知作物: ${crop}`);
  if (market === 'local' && !T.isMarketDay(now)) return { ok: false, reason: 'local_market_closed' }
;
  const mult = market === 'tourist' ? C.TOURIST_SEED_MULT : 1;
  return { ok: true, coin: Math.ceil(cfg.seedPrice * count * mult) };
}
module.exports = { plotState, canPlant, harvestYield, stealYield, sellPrice, seedPrice };
'use strict';
const C = require('./config');
function canSaveProfile(ctx) {
  const { nick, whisper } = ctx;
  if (nick !== undefined) {
    const len = nick.trim().length;
    if (len < C.LEN.nickname[0]) return { ok: false, reason: 'nick_too_short', field: 'nick' };
    if (len > C.LEN.nickname[1]) return { ok: false, reason: 'book_too_long', field: 'nick' };
  }
  if (whisper !== undefined && whisper.length > C.LEN.whisper[1]) {
    return { ok: false, reason: 'book_too_long', field: 'whisper' };
  }
  return { ok: true };
}
function canCheckIn(name) {
  if (typeof name !== 'string') return { ok: false, reason: 'bad_request', field: 'check_in_name' };
  const len = name.trim().length;
  if (len < C.LEN.checkInName[0] || len > C.LEN.checkInName[1]) {
    return { ok: false, reason: 'bad_request', field: 'check_in_name' };
  }
  return { ok: true };
}
function clampGiftValue(v) {
  const n = Math.floor(Number(v) || 0);
  return Math.max(0, Math.min(9999, n));
}
function canAdoptInviter(ctx) {
  const { existingInviter, selfOpenid, inviter, inviterExists } = ctx;
  if (!inviter) return false;
```

```
  if (existingInviter) return false;
  if (inviter === selfOpenid) return false;
  if (!inviterExists) return false;
  return true;
}
function sanitizeCoins(v) {
  if (typeof v !== 'number' || !isFinite(v)) return undefined;
  return Math.max(0, Math.min(999999, Math.floor(v)));
}
const GAME_ID_RE = /^[A-Za-z0-9_]{1,20}$/;
const GAMES_MAX_KEYS = 12;
function sanitizeGames(v) {
  if (typeof v !== 'object' || v === null || Array.isArray(v)) return undefined;
  const out = {};
  let count = 0;
  for (const key of Object.keys(v)) {
    if (count >= GAMES_MAX_KEYS) break;
    if (!GAME_ID_RE.test(key)) continue;
    const val = v[key];
    if (typeof val !== 'number' || !isFinite(val)) continue;
    out[key] = Math.max(0, Math.min(999, Math.floor(val)));
    count++;
  }
  return out;
}
function sanitizeDishes(v) {
  if (!Array.isArray(v)) return undefined;
  const seen = new Set();
  const out = [];
  for (const item of v) {
    if (typeof item !== 'number' || !Number.isInteger(item)) continue;
    if (item < 0 || item > 99) continue;
    if (seen.has(item)) continue;
    seen.add(item);
    out.push(item);
    if (out.length >= 100) break;
  }
  return out;
}
function mergeGames(existing, incoming) {
  return Object.assign({}, existing, incoming);
}
function mergeDishes(existing, incoming) {
  const set = new Set(existing);
  for (const n of incoming) set.add(n);
  return Array.from(set).sort((a, b) => a - b).slice(0, 100);
}
function sanitizeGiftAdd(v) {
  if (typeof v !== 'number' || !isFinite(v)) return null;
  const n = Math.floor(v);
```

```
  if (n < 1 || n > 999999) return null;
  return n;
}
module.exports = {
  canSaveProfile, canCheckIn, clampGiftValue, canAdoptInviter,
  sanitizeCoins, sanitizeGames, sanitizeDishes, sanitizeGiftAdd,
  mergeGames, mergeDishes,
};
'use strict';
const ZERO_WIDTH_RE = /[-- ]/g;
const VARIATION_SELECTOR_RE = /[-]|[\u{E0100}-\u{E01EF}]/gu;
const WS_RE = /\s+/g;
function toHalfWidth(s) {
  let out = '';
  for (const ch of s) {
    const code = ch.codePointAt(0);
    if (code === 0x3000) { out += ' '; continue; }
    if (code >= 0xFF01 && code <= 0xFF5E) { out += String.fromCodePoint(code - 0xFEE0); continue; }
    out += ch;
  }
  return out;
}
function normalize(raw) {
  let s = String(raw == null ? '' : raw);
  s = s.replace(VARIATION_SELECTOR_RE, '').replace(ZERO_WIDTH_RE, '');
  s = s.trim().replace(WS_RE, ' ');
  s = toHalfWidth(s);
  s = s.toLowerCase();
  s = s.trim().replace(WS_RE, ' ');
  return s;
}
const PUNCT_SYMBOL_RE = /[\s\p{P}\p{S}]/gu;
function stripForB(a) {
  return a.replace(PUNCT_SYMBOL_RE, '');
}
function buildMatcher(words) {
  const root = { next: new Map(), fail: null, end: false };
  let size = 0;
  for (const w of words || []) {
    if (!w) continue;
    let node = root;
    for (const ch of w) {
      let nxt = node.next.get(ch);
      if (!nxt) { nxt = { next: new Map(), fail: null, end: false }; node.next.set(ch, nxt); }
      node = nxt;
    }
    if (!node.end) size++;
    node.end = true;
  }
  const queue = [];
```

```
  for (const child of root.next.values()) { child.fail = root; queue.push(child); }
  let qi = 0;
  while (qi < queue.length) {
    const node = queue[qi++];
    for (const [ch, child] of node.next) {
      let f = node.fail;
      while (f !== root && !f.next.has(ch)) f = f.fail;
      const cand = f.next.get(ch);
      child.fail = (cand && cand !== child) ? cand : root;
      if (child.fail.end) child.end = true;
      queue.push(child);
    }
  }
  return {
    size,
    hasHit(text) {
      let node = root;
      for (const ch of String(text || '')) {
        while (node !== root && !node.next.has(ch)) node = node.fail;
        node = node.next.get(ch) || root;
        if (node.end) return true;
      }
      return false;
    },
  };
}
function checkText(text, opts) {
  const { sensitiveMatcher, reservedSet } = opts || {};
  const a = normalize(text);
  if (reservedSet && reservedSet.size && reservedSet.has(a)) return { ok: false, reason: 'reserved'
};
  if (sensitiveMatcher) {
    if (sensitiveMatcher.hasHit(a)) return { ok: false, reason: 'sensitive' };
    const b = stripForB(a);
    if (b !== a && sensitiveMatcher.hasHit(b)) return { ok: false, reason: 'sensitive' };
  }
  return { ok: true };
}
module.exports = { normalize, toHalfWidth, stripForB, buildMatcher, checkText };
'use strict';
const C = require('./config');
const T = require('./time');
const farm = require('./farm');
function canSteal(ctx) {
  const { thief, owner, plot, todayStealCount, todayStealOnOwner, guardUntil, guardRoll, now } = ctx
;
  if (thief === owner) return { ok: false, reason: 'own_plot' };
  const st = farm.plotState(plot, now);
  if (st.state !== 'ripe') return { ok: false, reason: 'not_ripe' };
  if ((plot.stolenCount || 0) >= 1) return { ok: false, reason: 'already_stolen' };
```

```
  if (todayStealCount >= C.STEAL_DAILY_LIMIT) return { ok: false, reason: 'daily_limit' };
  if (todayStealOnOwner >= C.STEAL_PER_HOUSE_LIMIT) return { ok: false, reason: 'house_limit' };
  if (guardUntil && now < guardUntil && guardRoll < C.GUARD_BLOCK_RATE) {
    return { ok: false, reason: 'guarded' };
  }
  return { ok: true, yield: farm.stealYield() };
}
function canBorrowGuard(todayBorrowCount) {
  if (todayBorrowCount >= C.GUARD_DAILY_BORROW) return { ok: false, reason: 'borrowed_today' };
  return { ok: true, durationMs: C.GUARD_DURATION_MS };
}
module.exports = { canSteal, canBorrowGuard };
'use strict';
const C = require('./config');
const T = require('./time');
const tree = require('./tree');
function canMakeThread(ctx) {
  const { maker, a, b, weekThreadCount, friends, blocked, pastThreads, now } = ctx;
  if (a === b) return { ok: false, reason: 'same_person' };
  if (maker === a || maker === b) return { ok: false, reason: 'cannot_include_self' };
  if (weekThreadCount >= C.THREAD_WEEKLY_QUOTA) return { ok: false, reason: 'weekly_quota' };
  if (friends.has(tree.pairKey(a, b))) return { ok: false, reason: 'already_friend' };
  if (blocked.has(tree.pairKey(a, b))) return { ok: false, reason: 'blocked' };
  const failed = pastThreads.filter(t =>
    tree.pairKey(t.a, t.b) === tree.pairKey(a, b) && t.maker === maker && t.status === 'broken');
  if (failed.length >= 1) return { ok: false, reason: 'already_tried' };
  return { ok: true, expireAt: now + C.THREAD_EXPIRE_MS };
}
function threadResult(thread, now) {
  if (thread.aAccepted === false || thread.bAccepted === false) return 'broken';
  if (thread.aAccepted === true && thread.bAccepted === true) return 'connected';
  if (now >= thread.expireAt) return 'broken';
  return 'pending';
}
function canDrawStick(lastDrawAt, now) {
  if (!lastDrawAt) return { ok: true };
  if (T.localDayStart(lastDrawAt) === T.localDayStart(now)) {
    return { ok: false, reason: 'drawn_today' };
  }
  return { ok: true };
}
function drawStick(ctx) {
  const { me, wishers, parentOf, familyOf, blocked, kept, returned, rand, now } = ctx;
  const returnedRecently = new Set(
    returned.filter(r => now - r.at < C.STICK_RETURN_COOLDOWN_MS).map(r => r.who)
  );
  const family = new Set(familyOf[me] || []);
  const keptSet = new Set(kept);
  const pool = wishers.filter(w => {
    if (w === me) return false;
```

```
    if (keptSet.has(w)) return false;
    if (returnedRecently.has(w)) return false;
    if (blocked.has(tree.pairKey(me, w))) return false;
    if (family.has(w)) return false;
    if (parentOf[me] === w || parentOf[w] === me) return false;
    return true;
  });
  if (pool.length === 0) return { ok: false, reason: 'no_stick_today' };
  const idx = Math.floor(rand * pool.length) % pool.length;
  return { ok: true, who: pool[idx] };
}
function isConnected(aKept, bKept, a, b) {
  return aKept.includes(b) && bKept.includes(a);
}
function findIntroducer(a, b, recommendsOf, parentOf) {
  const recA = (recommendsOf[a] || []).map(r => r.by);
  const recB = (recommendsOf[b] || []).map(r => r.by);
  const both = recA.filter(x => recB.includes(x));
  if (both.length) return { who: both[0], via: 'recommended_both' };
  if (recB.length) return { who: recB[0], via: 'recommended_them' };
  if (recA.length) return { who: recA[0], via: 'recommended_you' };
  const chain = (x) => { const out = []; let c = parentOf[x]; while (c && !out.includes(c)) { out.pu
sh(c); c = parentOf[c]; } return out; };
  const ca = chain(a), cb = chain(b);
  const common = ca.find(x => cb.includes(x));
  if (common) return { who: common, via: 'common_branch' };
  return { who: null, via: 'none' };
}
function canRecommend(by, target, existing) {
  if (by === target) return { ok: false, reason: 'self' };
  if (existing.some(r => r.by === by)) return { ok: false, reason: 'already_wrote' };
  if (existing.length >= 5) return { ok: false, reason: 'full' };
  return { ok: true };
}
module.exports = { canMakeThread, threadResult, canDrawStick, drawStick, isConnected, findIntroducer
, canRecommend };
'use strict';
const C = require('./config');
function localDate(ts) {
  return new Date(ts + C.TZ_OFFSET).toISOString().slice(0, 10);
}
function localDayStart(ts) {
  return Math.floor((ts + C.TZ_OFFSET) / C.DAY) * C.DAY - C.TZ_OFFSET;
}
function isMarketDay(ts) {
  return Number(localDate(ts).slice(8, 10)) % 2 === 1;
}
function isFloodSeason(ts) {
  const m = Number(localDate(ts).slice(5, 7));
  return m >= 6 && m <= 8;
```

```
}
function isMorningMarket(ts) {
  const h = new Date(ts + C.TZ_OFFSET).getUTCHours();
  return h >= 6 && h < 10;
}
function isNightTour(ts) {
  const d = new Date(ts + C.TZ_OFFSET);
  const mins = d.getUTCHours() * 60 + d.getUTCMinutes();
  return mins >= 19 * 60 + 30 && mins < 20 * 60 + 30;
}
function localWeekStart(ts) {
  const dayStart = localDayStart(ts);
  const dow = new Date(dayStart + C.TZ_OFFSET).getUTCDay();
  const back = (dow + 6) % 7;
  return dayStart - back * C.DAY;
}
module.exports = { localDate, localDayStart, isMarketDay, isFloodSeason, isMorningMarket, isNightTou
r, localWeekStart };
'use strict';
const C = require('./config');
const T = require('./time');
function lampProgress(totalDonated) {
  const lit = Math.min(C.LAMP_TOTAL, Math.floor(totalDonated / C.LAMP_COST));
  const intoCurrent = totalDonated - lit * C.LAMP_COST;
  return {
    lit,
    total: C.LAMP_TOTAL,
    allLit: lit >= C.LAMP_TOTAL,
    current: lit >= C.LAMP_TOTAL ? null : lit + 1,
    progress: lit >= C.LAMP_TOTAL ? null : `${intoCurrent} / ${C.LAMP_COST}`,
  };
}
function lampNames(donations) {
  const lamps = [];
  let bucket = [];
  let acc = 0;
  for (const d of donations) {
    let remain = d.count;
    while (remain > 0 && lamps.length < C.LAMP_TOTAL) {
      const need = C.LAMP_COST - acc;
      const put = Math.min(need, remain);
      bucket.push({ who: d.who, count: put });
      acc += put; remain -= put;
      if (acc >= C.LAMP_COST) {
        lamps.push(topN(bucket));
        bucket = []; acc = 0;
      }
    }
  }
  return lamps;
```

```
}
function topN(bucket) {
  const sum = {};
  for (const b of bucket) sum[b.who] = (sum[b.who] || 0) + b.count;
  return Object.entries(sum)
    .sort((a, b) => b[1] - a[1])
    .slice(0, C.LAMP_NAME_TOP)
    .map(([who, count]) => ({ who, count }));
}
function ballsOfDay(ts, spots, seedFn) {
  const date = T.localDate(ts);
  const out = [];
  const colors = ['red', 'yellow', 'blue', 'old'];
  for (let i = 0; i < C.BALL_COUNT; i++) {
    const r = seedFn(`${date}-ball-${i}`);
    out.push({ id: `${date}-${i}`, color: colors[i], spot: spots[Math.floor(r * spots.length) % spot
s.length] });
  }
  return out;
}
function canKick(who, ballId, todayKicks) {
  if (todayKicks.some(k => k.who === who && k.ballId === ballId)) {
    return { ok: false, reason: 'kicked_this_ball' };
  }
  return { ok: true };
}
function ballGoalReached(todayKickCount) {
  return todayKickCount >= C.BALL_GOAL;
}
function weeklyPaper(data, weekStart, weekEnd, issue) {
  const inWeek = (at) => at >= weekStart && at < weekEnd;
  const stealBy = {}, stolenFrom = {};
  for (const s of data.steals.filter(x => inWeek(x.at))) {
    stealBy[s.thief] = (stealBy[s.thief] || 0) + s.count;
    stolenFrom[s.owner] = (stolenFrom[s.owner] || 0) + s.count;
  }
  const saleBy = {};
  for (const s of data.sales.filter(x => inWeek(x.at))) saleBy[s.who] = (saleBy[s.who] || 0) + s.coi
n;
  const joins = data.joins.filter(x => inWeek(x.at));
  const byBranch = {};
  for (const j of joins) byBranch[j.parent] = (byBranch[j.parent] || 0) + 1;
  const petCount = {};
  for (const p of data.petEvents.filter(x => inWeek(x.at))) {
    petCount[p.pet] = petCount[p.pet] || {};
    petCount[p.pet][p.type] = (petCount[p.pet][p.type] || 0) + 1;
  }
  const best = (obj) => {
    const e = Object.entries(obj).sort((a, b) => b[1] - a[1])[0];
    return e ? { who: e[0], n: e[1] } : null;
```

```
  };
  const bestHouse = [...data.houseRatings].filter(h => h.count > 0)
    .sort((a, b) => b.avg - a.avg)[0] || null;
  return {
    issue,
    weekStart, weekEnd,
    columns: {
      手最长的: { top: best(stealBy), victim: best(stolenFrom) },
      地最肥的: best(saleBy),
      新来的: { count: joins.length, byBranch },
      最安逸的屋: bestHouse,
      猫猫狗狗的一周: petCount,
    },
    lamps: data.lamps,
  };
}
module.exports = { lampProgress, lampNames, ballsOfDay, canKick, ballGoalReached, weeklyPaper };
'use strict';
const C = require('./config');
const T = require('./time');
function inviteQuota(usedCount, activeWeeks) {
  const total = C.INVITE_QUOTA_BASE + C.INVITE_QUOTA_PER_WEEK * Math.max(0, activeWeeks);
  return { total, used: usedCount, left: Math.max(0, total - usedCount) };
}
function pairKey(a, b) { return a < b ? `${a}|${b}` : `${b}|${a}`; }
function relation(a, b, parentOf, friends) {
  if (a === b) return 'self';
  if (parentOf[a] === b || parentOf[b] === a) return 'same_branch';
  if (friends.has(pairKey(a, b))) return 'friend';
  if (parentOf[a] && parentOf[a] === parentOf[b]) return 'sibling';
  return 'none';
}
function canRequestFriend(a, b, parentOf, friends, friendsOf) {
  const rel = relation(a, b, parentOf, friends);
  if (rel === 'self') return { ok: false, reason: 'self' };
  if (rel === 'friend' || rel === 'same_branch') return { ok: false, reason: 'already_friend' };
  if (rel === 'sibling') return { ok: true, via: 'sibling' };
  for (const f of (friendsOf[a] || [])) {
    if ((friendsOf[f] || []).includes(b)) return { ok: true, via: 'friend_of_friend' };
  }
  return { ok: false, reason: 'too_far' };
}
function rankSuggestions(candidates) {
  const score = (s) =>
    (s.sibling ? 1000 : 0) +
    (s.mutualFriends || 0) * 100 +
    (s.stoleFromMe ? 50 : 0) +
    (s.sameWeek ? 10 : 0);
  return candidates
    .map(c => ({ ...c, _score: score(c.signals) }))
```

```
    .filter(c => c._score > 0)
    .sort((x, y) => y._score - x._score)
    .slice(0, C.SUGGEST_LIMIT);
}
function buildTree(members, parentOf) {
  const nodes = {};
  for (const m of members) nodes[m] = { id: m, children: [] };
  const roots = [];
  for (const m of members) {
    const p = parentOf[m];
    if (p && nodes[p]) nodes[p].children.push(nodes[m]);
    else roots.push(nodes[m]);
  }
  return roots;
}
module.exports = { inviteQuota, relation, canRequestFriend, rankSuggestions, buildTree, pairKey };
const cloud = require('wx-server-sdk');
const canvas = require('./rules/canvas');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const CACHE_MS = 5 * 60 * 1000;
let cache = null;
exports.main = async (event) => {
  const { fileID } = event || {};
  const offset = event && event.offset !== undefined ? event.offset : 0;
  if (!canvas.canReadCanvas(fileID)) return { ok: false, reason: 'bad_request' };
  try {
    const now = Date.now();
    if (!cache || cache.fileID !== fileID || now - cache.at > CACHE_MS) {
      const res = await cloud.downloadFile({ fileID });
      cache = { fileID, buf: res.fileContent, at: now };
    }
    const buf = cache.buf;
    const r = canvas.chunkRange(buf.length, offset);
    if (!r.ok) return r;
    const part = buf.subarray(r.start, r.end);
    return { ok: true, total: buf.length, offset: r.start, size: part.length, data: part.toString('b
ase64'), done: r.done };
  } catch (e) {
    const why = String((e && (e.errMsg || e.message)) || e).slice(0, 200);
    console.error('canvasFile 读文件出错：', why);
    cache = null;
    return { ok: false, reason: 'download_failed', detail: why };
  }
};
const cloud = require('wx-server-sdk');
const banquet = require('./rules/banquet');
const C = require('./rules/config');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
exports.main = async (event) => {
```

```
  const { tableId, chairNo } = event || {};
  if (!tableId || !chairNo) return { ok: false, reason: 'bad_request' };
  if (tableId === C.BANQUET_BENCH_ID) return { ok: false, reason: 'bad_request' };
  const mainId = banquet.mainTableIdOf(C.BANQUET_TABLE_IDS);
  let isVip = false;
  if (mainId && tableId === mainId) {
    const doc = await db.collection('config').doc('banquet').get().catch(() => null);
    const vipIds = (doc && doc.data && doc.data.vip_openids) || [];
    isVip = vipIds.includes(cloud.getWXContext().OPENID);
  }
  const target = banquet.checkSeatTarget({ tableId, chairNo, tableIds: C.BANQUET_TABLE_IDS, mainTabl
eId: mainId, isVip });
  if (!target.ok) return target;
  const { OPENID } = cloud.getWXContext();
  const now = Date.now();
  const mySeatDoc = await db.collection('seat').doc(OPENID).get().catch(() => null);
  const mySeat = mySeatDoc && mySeatDoc.data;
  if (!mySeat) return { ok: false, reason: 'no_seat' };
  const { data: atTarget } = await db.collection('seat')
    .where({ table_id: tableId, chair_no: chairNo }).get();
  const occupant = atTarget.length ? atTarget[0].openid : null;
  const result = banquet.canChangeSeat({ occupant });
  if (!result.ok) return result;
  const satTables = mySeat.sat_tables && mySeat.sat_tables.length ? mySeat.sat_tables.slice() : [myS
eat.table_id];
  if (!satTables.includes(tableId)) satTables.push(tableId);
  await db.collection('seat').doc(OPENID).update({
    data: { table_id: tableId, chair_no: chairNo, sat_at: now, sat_tables: satTables },
  });
  return { ok: true, data: { table_id: tableId, chair_no: chairNo } };
};
const cloud = require('wx-server-sdk');
const banquet = require('./rules/banquet');
const C = require('./rules/config');
const { fetchAllPages } = require('./lib/db-page');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const _ = db.command;
const TABLE_IDS = C.BANQUET_TABLE_IDS;
const RESERVED_IDS = new Set(TABLE_IDS.slice(0, C.BANQUET_RESERVED_TABLES));
const MAIN_ID = banquet.mainTableIdOf(TABLE_IDS);
const BENCH = C.BANQUET_BENCH_ID;
const PAGE_OPTS = { orderBy: [['_id', 'asc']], maxTotal: C.BANQUET_PAGE_MAX_TOTAL };
async function loadVip() {
  const doc = await db.collection('config').doc('banquet').get().catch(() => null);
  const ids = (doc && doc.data && doc.data.vip_openids) || [];
  return new Set(ids);
}
async function loadTables() {
  const seats = await fetchAllPages(() => db.collection('seat').where({ table_id: _.neq(BENCH) }), P
```

```
AGE_OPTS);
  const byTable = {};
  for (const s of seats) (byTable[s.table_id] = byTable[s.table_id] || []).push(s);
  return {
    tables: TABLE_IDS.map(id => ({
      id,
      reserved: RESERVED_IDS.has(id),
      branch: byTable[id] && byTable[id][0] ? byTable[id][0].inviter || null : null,
      seats: (byTable[id] || []).map(s => s.openid),
    })),
    mainTable: MAIN_ID ? { id: MAIN_ID, seats: (byTable[MAIN_ID] || []).map(s => s.openid) } : null,
    byTable,
  };
}
exports.main = async () => {
  const { OPENID } = cloud.getWXContext();
  const now = Date.now();
  const residentDoc = await db.collection('resident').doc(OPENID).get().catch(() => null);
  if (!residentDoc || !residentDoc.data) return { ok: false, reason: 'no_resident' };
  const inviter = residentDoc.data.inviter || null;
  let mySeat = await db.collection('seat').doc(OPENID).get().catch(() => null);
  mySeat = mySeat && mySeat.data;
  const firstVisit = !mySeat;
  let tableWasEmpty = false;
  let vip = null;
  if (!mySeat) {
    let tables, mainTable, byTable;
    [{ tables, mainTable, byTable }, vip] = await Promise.all([loadTables(), loadVip()]);
    const assigned = banquet.assignSeat({
      inviter, vip, tables, me: OPENID, benchOnlyWhenFull: C.BANQUET_BENCH_ONLY_WHEN_FULL, mainTable
,
    });
    if (assigned.bench) {
      mySeat = {
        _id: OPENID, openid: OPENID, table_id: BENCH, chair_no: null, inviter,
        sat_at: now, changed_on: null, sat_tables: [], last_visit_at: now,
      };
    } else {
      tableWasEmpty = !(byTable[assigned.tableId] || []).length;
      const used = new Set((byTable[assigned.tableId] || []).map(s => s.chair_no));
      let chairNo = 1;
      while (used.has(chairNo)) chairNo++;
      mySeat = {
        _id: OPENID, openid: OPENID, table_id: assigned.tableId, chair_no: chairNo, inviter,
        sat_at: now, changed_on: null, sat_tables: [assigned.tableId], last_visit_at: now,
      };
    }
    await db.collection('seat').add({ data: mySeat });
  }
  const since = mySeat.last_visit_at || mySeat.sat_at;
```

```
  const toastSince = Math.max(since, mySeat.toast_seen_at || 0);
  const satTables = mySeat.sat_tables && mySeat.sat_tables.length ? mySeat.sat_tables : [mySeat.tabl
e_id];
  const [toasts, bookLines, visitorSeats] = await Promise.all([
    fetchAllPages(() => db.collection('toast').where({ to: OPENID, created_at: _.gt(toastSince) }),
PAGE_OPTS),
    fetchAllPages(() => db.collection('guestbook').where({ table_id: _.in(satTables), created_at: _.
gt(since) }), PAGE_OPTS),
    firstVisit ? Promise.resolve([])
      : fetchAllPages(() => db.collection('seat').where({ last_visit_at: _.gt(since) }), PAGE_OPTS),
  ]);
  const digest = banquet.arrivalDigest({
    toasts: toasts.map(t => ({ from: t.from, at: t.created_at })),
    bookLines: bookLines.map(l => ({ tableId: l.table_id, at: l.created_at })),
    since,
  });
  const myTableLines = bookLines
    .filter(l => l.table_id === mySeat.table_id && l.openid && l.openid !== OPENID)
    .map(l => ({ from: l.openid, at: l.created_at, text: l.text || '' }))
    .sort((a, b) => b.at - a.at);
  const visitors = visitorSeats
    .filter(s => s.openid !== OPENID)
    .map(s => ({ openid: s.openid, at: s.last_visit_at }))
    .sort((a, b) => b.at - a.at);
  const card = banquet.arrivalCard({
    me: OPENID,
    since,
    toasts: digest.toasts,
    bookLines: myTableLines,
    visitors,
  });
  const arrival = {
    first_visit: firstVisit,
    table_was_empty: tableWasEmpty,
    rows: card.rows,
    hasNews: card.hasNews,
    visitors: visitors.slice(0, C.BANQUET_ARRIVAL_LIST_MAX),
    book_lines: myTableLines.slice(0, C.BANQUET_ARRIVAL_LIST_MAX),
  };
  if (mySeat.last_visit_at !== now) {
    await db.collection('seat').doc(OPENID).update({ data: { last_visit_at: now } });
  }
  const allSeats = await fetchAllPages(() => db.collection('seat'), PAGE_OPTS);
  const byTableNow = {};
  const bench = [];
  for (const s of allSeats) {
    if (s.table_id === BENCH) bench.push(s.openid);
    else (byTableNow[s.table_id] = byTableNow[s.table_id] || []).push({ openid: s.openid, chair_no:
s.chair_no, sat_at: s.sat_at || 0 });
  }
```

```
  const ids = new Set(allSeats.map(s => s.openid));
  for (const t of digest.toasts) ids.add(t.from);
  for (const r of card.rows) if (r.who) ids.add(r.who);
  const names = {};
  async function loadNames(idArray) {
    for (let i = 0; i < idArray.length; i += 100) {
      const { data: rs } = await db.collection('resident').where({ _id: _.in(idArray.slice(i, i + 10
0)) })
        .field({ _id: true, nick: true, avatar: true, inviter: true }).get();
      for (const r of rs) {
        names[r._id] = { nick: r.nick || '', avatar: r.avatar === undefined ? -1 : r.avatar, inviter
: r.inviter || null };
      }
    }
  }
  await loadNames(Array.from(ids));
  const missingInviterIds = Array.from(new Set(
    Object.values(names).map(n => n.inviter).filter(id => id && !names[id]),
  ));
  if (missingInviterIds.length) await loadNames(missingInviterIds);
  const rd = residentDoc.data;
  const mine = {
    coins: typeof rd.coins === 'number' ? rd.coins : 0,
    games: (rd.games && typeof rd.games === 'object' && !Array.isArray(rd.games)) ? rd.games : {},
    dishes: Array.isArray(rd.dishes) ? rd.dishes : [],
    gift_coins: typeof rd.gift_coins === 'number' ? rd.gift_coins : 0,
  };
  const data = {
      me: { table_id: mySeat.table_id, chair_no: mySeat.chair_no },
      tables: (MAIN_ID ? [{ id: MAIN_ID, main: true, reserved: false, seats: byTableNow[MAIN_ID] ||
[] }] : [])
        .concat(TABLE_IDS.map(id => ({ id, reserved: RESERVED_IDS.has(id), seats: byTableNow[id] ||
[] }))),
      bench,
      digest,
      arrival,
      names,
      mine,
  };
  if (MAIN_ID) data.mainSeatOk = (vip || await loadVip()).has(OPENID);
  return { ok: true, data };
};
const cloud = require('wx-server-sdk');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const _ = db.command;
const MAX_TOASTS_PER_PULSE = 10;
const OVERLAP_MS = 3000;
exports.main = async (event) => {
  const { since } = event || {};
```

```
  const { OPENID } = cloud.getWXContext();
  const now = Date.now();
  const sinceValid = typeof since === 'number' && isFinite(since) && since > 0;
  if (!sinceValid) return { ok: true, now, toasts: [], names: {} };
  try {
    const { data: rows } = await db.collection('toast')
      .where({ to: OPENID, created_at: _.gt(since - OVERLAP_MS) })
      .orderBy('created_at', 'asc')
      .limit(MAX_TOASTS_PER_PULSE)
      .get();
    const toasts = rows
      .filter(r => r.created_at <= now)
      .map(r => ({ from: r.from, created_at: r.created_at }));
    if (!toasts.length) return { ok: true, now, toasts: [], names: {} };
    const fromIds = Array.from(new Set(toasts.map(t => t.from)));
    const names = {};
    for (const id of fromIds) names[id] = { nick: '', avatar: -1 };
    const { data: residents } = await db.collection('resident')
      .where({ _id: _.in(fromIds) })
      .field({ _id: true, nick: true, avatar: true })
      .get();
    for (const r of residents) {
      names[r._id] = { nick: r.nick || '', avatar: r.avatar === undefined ? -1 : r.avatar };
    }
    const seenAt = Math.max(...toasts.map(t => t.created_at));
    await db.collection('seat').doc(OPENID).update({ data: { toast_seen_at: seenAt } }).catch(() =>
null);
    return { ok: true, now, toasts, names };
  } catch (e) {
    return { ok: false, reason: 'network' };
  }
};
const cloud = require('wx-server-sdk');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
exports.main = async () => {
  const existing = await db.collection('config').doc('banquet').get().catch(() => null);
  if (existing && existing.data) {
    return { ok: true, data: { config_banquet: 'existed' } };
  }
  await db.collection('config').add({ data: { _id: 'banquet', vip_openids: [] } });
  return { ok: true, data: { config_banquet: 'created' } };
};
const cloud = require('wx-server-sdk');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
exports.main = async () => {
    const { OPENID, APPID } = cloud.getWXContext();
    return { openid: OPENID, appid: APPID };
};
const cloud = require('wx-server-sdk');
```

```
const C = require('./rules/config');
const { fetchAllPages } = require('./lib/db-page');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const _ = db.command;
exports.main = async (event) => {
  const { table } = event || {};
  if (!table) return { ok: false, reason: 'bad_request' };
  const rowsDesc = await fetchAllPages(
    () => db.collection('guestbook').where({ table_id: table }),
    { orderBy: [['created_at', 'desc'], ['_id', 'desc']], maxTotal: C.BANQUET_BOOK_READ_MAX },
  );
  const rows = rowsDesc.slice().reverse();
  const ids = Array.from(new Set(rows.map(l => l.openid)));
  const names = {};
  for (let i = 0; i < ids.length; i += 100) {
    const { data: rs } = await db.collection('resident').where({ _id: _.in(ids.slice(i, i + 100)) })
      .field({ _id: true, nick: true, avatar: true }).get();
    for (const r of rs) names[r._id] = { nick: r.nick || '', avatar: r.avatar === undefined ? -1 : r
.avatar };
  }
  return {
    ok: true,
    data: {
      lines: rows.map(l => ({ openid: l.openid, text: l.text, at: l.created_at })),
      names,
    },
  };
};
const fs = require('fs');
const path = require('path');
const cloud = require('wx-server-sdk');
const resident = require('./rules/resident');
const sensitive = require('./rules/sensitive');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const FIELDS = ['nick', 'avatar', 'inviter', 'gift_value', 'whisper', 'coins', 'games', 'dishes', 'g
ift_coins', 'gift_add', 'check_in_name'];
function readWordFile(name) {
  const file = path.join(__dirname, 'words', name);
  if (!fs.existsSync(file)) return [];
  return fs.readFileSync(file, 'utf8').split(/\r?\n/).map(s => s.trim()).filter(Boolean);
}
const SENSITIVE_MATCHER = sensitive.buildMatcher(readWordFile('base.txt'));
const RESERVED_SET = new Set(readWordFile('reserved.txt').map(sensitive.normalize));
async function isVip(openid) {
  const doc = await db.collection('config').doc('banquet').get().catch(() => null);
  const ids = (doc && doc.data && doc.data.vip_openids) || [];
  return ids.indexOf(openid) !== -1;
}
```

```
async function checkName(text, ownNames, openid) {
  const wordCheck = sensitive.checkText(text, { sensitiveMatcher: SENSITIVE_MATCHER, reservedSet: RE
SERVED_SET });
  if (wordCheck.ok) return wordCheck;
  if (wordCheck.reason === 'reserved') {
    const n = sensitive.normalize(text);
    const own = ownNames.some(x => typeof x === 'string' && sensitive.normalize(x) === n);
    if (own || await isVip(openid)) return { ok: true };
  }
  return wordCheck;
}
exports.main = async (event) => {
  const { OPENID } = cloud.getWXContext();
  const now = Date.now();
  const fields = {};
  for (const k of FIELDS) {
    if (event && event[k] !== undefined) fields[k] = event[k];
  }
  if (Object.keys(fields).length === 0) return { ok: false, reason: 'bad_request' };
  const check = resident.canSaveProfile({ nick: fields.nick, whisper: fields.whisper });
  if (!check.ok) return check;
  if (fields.check_in_name !== undefined) {
    const ci = resident.canCheckIn(fields.check_in_name);
    if (!ci.ok) return ci;
    fields.check_in_name = fields.check_in_name.trim();
  }
  const existing = await db.collection('resident').doc(OPENID).get().catch(() => null);
  const existingData = existing && existing.data;
  if (fields.nick !== undefined) {
    const nickCheck = await checkName(fields.nick, [existingData && existingData.nick], OPENID);
    if (!nickCheck.ok) return nickCheck;
  }
  if (fields.check_in_name !== undefined) {
    const nameCheck = await checkName(fields.check_in_name, [existingData && existingData.nick, fiel
ds.nick], OPENID);
    if (!nameCheck.ok) return nameCheck;
    fields.checked_in_at = (existingData && existingData.checked_in_at) || now;
  }
  if (fields.gift_value !== undefined) fields.gift_value = resident.clampGiftValue(fields.gift_value
);
  if (fields.coins !== undefined) {
    const v = resident.sanitizeCoins(fields.coins);
    if (v === undefined) delete fields.coins; else fields.coins = v;
  }
  if (fields.gift_coins !== undefined) {
    const v = resident.sanitizeCoins(fields.gift_coins);
    if (v === undefined) delete fields.gift_coins; else fields.gift_coins = v;
  }
  if (fields.games !== undefined) {
    const v = resident.sanitizeGames(fields.games);
```

```
    if (v === undefined) {
      delete fields.games;
    } else {
      const existingGames = (existingData && existingData.games && typeof existingData.games === 'ob
ject'
        && !Array.isArray(existingData.games)) ? existingData.games : {};
      fields.games = resident.mergeGames(existingGames, v);
    }
  }
  if (fields.dishes !== undefined) {
    const v = resident.sanitizeDishes(fields.dishes);
    if (v === undefined) {
      delete fields.dishes;
    } else {
      const existingDishes = Array.isArray(existingData && existingData.dishes) ? existingData.dishe
s : [];
      fields.dishes = resident.mergeDishes(existingDishes, v);
    }
  }
  let giftAddCoins = null;
  if (fields.gift_add !== undefined) {
    giftAddCoins = resident.sanitizeGiftAdd(fields.gift_add);
    delete fields.gift_add;
  }
  if (fields.inviter !== undefined) {
    if (!fields.inviter) {
      delete fields.inviter;
    } else {
      const existingInviter = (existingData && existingData.inviter) || null;
      let inviterExists = false;
      if (fields.inviter !== OPENID) {
        const inviterDoc = await db.collection('resident').doc(fields.inviter).get().catch(() => nul
l);
        inviterExists = !!(inviterDoc && inviterDoc.data);
      }
      const allowed = resident.canAdoptInviter({
        existingInviter, selfOpenid: OPENID, inviter: fields.inviter, inviterExists,
      });
      if (!allowed) delete fields.inviter;
    }
  }
  if (existingData) {
    if (Object.keys(fields).length > 0) {
      await db.collection('resident').doc(OPENID).update({ data: fields });
    }
  } else {
    await db.collection('resident').add({
      data: { _id: OPENID, openid: OPENID, created_at: now, checked_in_at: null, ...fields },
    });
  }
```

```
  if (giftAddCoins !== null) {
    const nick = fields.nick !== undefined ? fields.nick : ((existingData && existingData.nick) || '
');
    try {
      await db.collection('gift_coin').add({
        data: { openid: OPENID, nick, coins: giftAddCoins, created_at: now },
      });
    } catch (e) {
      await db.collection('resident').doc(OPENID)
        .update({ data: { gift_log: db.command.push([{ coins: giftAddCoins, created_at: now }]) } })
        .catch(() => null);
    }
  }
  const checkedIn = !!(fields.checked_in_at || (existingData && existingData.checked_in_at));
  return { ok: true, checked_in: checkedIn };
};
const cloud = require('wx-server-sdk');
const banquet = require('./rules/banquet');
const T = require('./rules/time');
const C = require('./rules/config');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
exports.main = async (event) => {
  const { to } = event || {};
  if (!to) return { ok: false, reason: 'bad_request' };
  const { OPENID: from } = cloud.getWXContext();
  const now = Date.now();
  const today = T.localDate(now);
  const { data: mine } = await db.collection('toast').where({ from, date: today }).get();
  const todayToastCount = mine.length;
  const todayToastToCount = mine.filter(t => t.to === to).length;
  const result = banquet.canToast({ from, to, todayToastCount, todayToastToCount, testMode: C.BANQUE
T_TEST_MODE });
  if (!result.ok) return result;
  await db.collection('toast').add({ data: { from, to, date: today, created_at: now } });
  return { ok: true };
};
const cloud = require('wx-server-sdk');
const banquet = require('./rules/banquet');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
async function tableOf(openid) {
  const doc = await db.collection('seat').doc(openid).get().catch(() => null);
  return (doc && doc.data && doc.data.table_id) || null;
}
async function exists(coll, from, to) {
  const { data } = await db.collection(coll).where({ from, to }).limit(1).get();
  return data.length > 0;
}
exports.main = async (event) => {
```

```
  const { to } = event || {};
  if (!to || typeof to !== 'string') return { ok: false, reason: 'bad_request' };
  const { OPENID: from } = cloud.getWXContext();
  const now = Date.now();
  try {
    const alreadySent = await exists('friend_request', from, to);
    const fromTableId = await tableOf(from);
    const toTableId = await tableOf(to);
    const toasted = await exists('toast', from, to) || await exists('toast', to, from);
    const result = banquet.canWantToMeet({ from, to, alreadySent, fromTableId, toTableId, toasted })
;
    if (!result.ok) return result;
    await db.collection('friend_request').add({
      data: { from, to, created_at: now, status: 'pending', via: result.via },
    });
    return { ok: true };
  } catch (e) {
    console.error('wantToMeet 数据库出错：', e && e.message);
    return { ok: false, reason: 'network' };
  }
};
const fs = require('fs');
const path = require('path');
const cloud = require('wx-server-sdk');
const banquet = require('./rules/banquet');
const sensitive = require('./rules/sensitive');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
function readWordFile(name) {
  const file = path.join(__dirname, 'words', name);
  if (!fs.existsSync(file)) return [];
  return fs.readFileSync(file, 'utf8').split(/\r?\n/).map(s => s.trim()).filter(Boolean);
}
const SENSITIVE_MATCHER = sensitive.buildMatcher(readWordFile('base.txt'));
exports.main = async (event) => {
  const { table, text } = event || {};
  if (!table || typeof text !== 'string') return { ok: false, reason: 'bad_request' };
  const { OPENID } = cloud.getWXContext();
  const now = Date.now();
  const mySeatDoc = await db.collection('seat').doc(OPENID).get().catch(() => null);
  const mySeat = mySeatDoc && mySeatDoc.data;
  if (!mySeat) return { ok: false, reason: 'no_seat' };
  const result = banquet.canWriteBook({ table, mySeatTableId: mySeat.table_id, text });
  if (!result.ok) return result;
  const wordCheck = sensitive.checkText(text, { sensitiveMatcher: SENSITIVE_MATCHER });
  if (!wordCheck.ok) return wordCheck;
  await db.collection('guestbook').add({ data: { table_id: table, openid: OPENID, text, created_at:
now } });
  return { ok: true };
};
```